

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM
KHOA CÔNG NGHỆ THÔNG TIN



HCMUTE

MÔN HỌC : MẬT MÃ ỨNG DỤNG

BÁO CÁO CUỐI KÌ

HỌC KÌ II – 2025-2026

**CLOUD API-BASED NETWORK APPLICATION SECURITY FOR ONLINE
EDUCATION SERVICES**

GVHD: TS. Nguyễn Ngọc Tụ

SVTH:

Phạm Hồng Anh 24162008

Nguyễn Lê Hữu Chí 24162012

Phạm Hữu Ngọc 24162081

Mã lớp học: CRYP331980_02

Thành phố Hồ Chí Minh, tháng 6 năm 2026

Trường ĐH Công Nghệ Kỹ Thuật TP.HCM

Khoa: Công nghệ thông tin

ĐỀ CƯƠNG CHI TIẾT ĐỒ ÁN MÔN HỌC

Họ và tên sinh viên thực hiện 1: **Phạm Hồng Anh** MSSV 1: **24162008**

Họ và tên sinh viên thực hiện 2: **Nguyễn Lê Hữu Chí** MSSV 2: **24162012**

Họ và tên sinh viên thực hiện 3: **Phạm Hữu Ngọc** MSSV 3: **24162081**

Thời gian làm đồ án môn học: Học kỳ II năm học 2025-2026

Môn học: **Mật mã ứng dụng**

Tên đề tài: **Cloud API-Based Network Application Security for Small Company Services**

Giáo viên hướng dẫn: **TS. Nguyễn Ngọc Tụ**

Nhiệm vụ của đồ án môn học:

Lý thuyết:

- RESTful API
- NextJS
- JWT (JSON Web Token)
- OAuth2 cơ bản
- AES Encryption
- Password Hashing (bcrypt)
- HTTPS/TLS
- OWASP API Security Top 10
- Ngôn ngữ Java, Spring Boot, Spring Security
- Swagger/OpenAPI

Thực hành:

- Sử dụng Spring Boot xây dựng Backend (RESTful API web service).
- Sử dụng NextJS xây dựng giao diện người dùng.
- Sử dụng bcrypt để mã hóa mật khẩu.
- Sử dụng AES để mã hóa dữ liệu nhạy cảm khi lưu trữ.
- Sử dụng HTTPS/TLS để bảo vệ kết nối.
- Sử dụng Swagger/OpenAPI để hỗ trợ kiểm thử và tài liệu API.
- Kiểm thử bảo mật API bằng Postman và OWASP ZAP.
- Dockerize và triển khai hệ thống trên internet.

LỜI CẢM ƠN

Nhóm em xin chân thành cảm ơn Thầy Nguyễn Ngọc Tự, người đã trực tiếp hướng dẫn, định hướng và tận tình chỉ bảo chúng em trong suốt quá trình học tập cũng như thực hiện đề tài nghiên cứu về Mật mã học này. Những kiến thức chuyên môn sâu sắc, sự tận tâm và những góp ý quý báu của Thầy không chỉ giúp chúng em tháo gỡ những vướng mắc trong kỹ thuật triển khai, mà còn giúp nhóm em hoàn thiện tư duy logic để hoàn thành project một cách trọn vẹn nhất. Dù đã dành nhiều tâm huyết và cố gắng, song bài báo cáo chắc chắn khó tránh khỏi những thiếu sót thiếu sót nhất định. Nhóm em rất mong tiếp tục nhận được những nhận xét, đóng góp từ Thầy để nội dung đề tài được hoàn thiện hơn. Nhóm em xin kính chúc Thầy luôn mạnh khỏe, hạnh phúc và gặt hái được nhiều thành công hơn nữa trong sự nghiệp giảng dạy và nghiên cứu của mình!

This image shows a full page of primary-ruled paper. It features 20 evenly spaced horizontal dotted lines across the entire width of the page, providing a guide for handwriting practice. The background is white, and there are no margins or other markings present.

KÝ TÊN

MỤC LỤC

Chương 1. Tổng quan đề tài	1
1.1. Đặt vấn đề.....	1
1.2. Bối cảnh thực tiễn của bài toán	1
1.2.1. Ngữ cảnh hệ thống và ranh giới tin cậy.....	2
1.2.2. Tài sản cần bảo vệ	3
1.2.3. Tác nhân đe dọa và kịch bản tấn công chính.....	4
1.2.4. Đánh giá rủi ro trọng yếu.....	5
1.3. Lý do chọn đề tài	7
1.4. Mục tiêu của đề tài.....	7
1.4.1. Mục tiêu tổng quát.....	7
1.4.2. Mục tiêu cụ thể	8
1.4.3. Mục tiêu bảo mật	9
1.5. Câu hỏi nghiên cứu và định hướng giải quyết.....	10
1.6. Đối tượng và phạm vi nghiên cứu	11
1.6.1. Đối tượng nghiên cứu	11
1.6.2. Phạm vi triển khai.....	11
1.7. Phương pháp thực hiện.....	12
1.7.1. Nghiên cứu lý thuyết	12
1.7.2. Lựa chọn nền tảng kỹ thuật và xác định phạm vi xây dựng.....	12
1.7.3. Thiết kế và hiện thực hệ thống	13
1.7.4. Kiểm thử và đánh giá.....	13
1.8. Công cụ và môi trường sử dụng	13
1.9. Kết quả mong đợi của đề tài	14
1.10. Ý nghĩa khoa học và thực tiễn	15
1.11. Bố cục báo cáo.....	15

Chương 2. Cơ sở lý thuyết và công nghệ sử dụng.....	16
2.1. Một số khái niệm nền tảng trong an toàn thông tin.....	16
2.2. Phân biệt các cơ chế mật mã sử dụng trong đề tài	16
2.3. Tổng quan về RESTful API	17
2.4. Xác thực và phân quyền trong hệ thống API	18
2.5. JWT và Bearer Token.....	18
2.5.1. Cấu trúc JWT.....	18
2.5.2. Vai trò của Bearer Token	19
2.5.3. Các rủi ro cần kiểm soát	19
2.5.4. Refresh token.....	19
2.6. Băm mật khẩu bằng bcrypt.....	19
2.7. Mã hóa dữ liệu nhạy cảm bằng AES-GCM.....	20
2.7.1. Vì sao cần mã hóa dữ liệu lưu trữ	20
2.7.2. Đặc điểm của AES-GCM	20
2.7.3. AAD trong AES-GCM.....	21
2.8. HMAC-SHA256.....	21
2.9. HTTPS/TLS.....	22
2.10. RBAC, ownership check và chống BOLA/IDOR.....	22
2.11. Excessive Data Exposure và DTO response	23
2.12. Rate limiting và audit logging	23
2.12.1. Rate limiting	23
2.12.2. Audit log.....	24
2.13. Swagger/OpenAPI.....	24
2.14. OWASP API Security Top 10	25
2.15. Docker và triển khai đóng gói	25
Chương 3. Phân tích, thiết kế và triển khai hệ thống	27

3.1. Định hướng cấu trúc kỹ thuật của hệ thống.....	27
3.2. Mô tả bài toán nghiệp vụ	28
3.3. Yêu cầu hệ thống	29
3.3.1. Yêu cầu chức năng	29
3.3.2. Yêu cầu phi chức năng	30
3.3.3. Yêu cầu bảo mật	30
3.4. Thiết kế tác nhân và use case	31
3.4.1. Tác nhân	31
3.4.2. Use case chính	31
3.5. Kiến trúc giải pháp bảo mật.....	32
3.5.1. Quan điểm thiết kế giải pháp.....	33
3.5.2. Tầng mạng (net)	35
3.5.3. Tầng host / service (host)	36
3.5.4. Tầng logic ứng dụng (logic)	37
3.5.5. Tầng luồng dữ liệu bảo mật (data flow)	38
3.6. Thiết kế phân quyền và scope	40
3.6.1. Role và phạm vi quyền	40
3.6.2. Kiểm tra ở SecurityConfig	40
3.6.3. Kiểm tra ownership ở AuthorizationService	41
3.7. Thiết kế cơ sở dữ liệu	42
3.7.1. Bảng users.....	42
3.7.2. Bảng courses.....	43
3.7.3. Bảng lessons	43
3.7.4. Bảng enrollments.....	44
3.7.5. Bảng certificates	44
3.7.6. Bảng refresh_tokens	44

3.7.7. Bảng audit_logs	45
3.8. Thiết kế nhóm API chính	45
3.8.1. Nhóm Auth API.....	45
3.8.2. Nhóm Course API	46
3.8.3. Nhóm Lesson API	46
3.8.4. Nhóm Enrollment API.....	46
3.8.5. Nhóm Certificate API.....	47
3.8.6. Nhóm Admin API.....	47
3.8.7. Nhóm System API.....	48
3.9. Thiết kế các luồng nghiệp vụ và bảo mật	49
3.9.1. Luồng đăng ký và lưu mật khẩu	49
3.9.2. Luồng đăng nhập và phát hành JWT	49
3.9.3. Luồng xác thực request protected.....	50
3.9.4. Luồng refresh token.....	50
3.9.5. Luồng quên mật khẩu và đặt lại mật khẩu.....	51
3.9.6. Luồng yêu cầu ghi danh và phê duyệt ghi danh	51
3.9.7. Luồng chống BOLA/IDOR	52
3.9.8. Luồng xem bài học	53
3.10. Triển khai các kỹ thuật mật mã trong mã nguồn	53
3.10.1. Triển khai JWT trong JwtService.....	53
3.10.2. Triển khai bcrypt trong SecurityConfig và AuthService.....	53
3.10.3. Triển khai AES-GCM trong EncryptionService	54
3.10.4. Triển khai kiểm soát ghi danh trong EnrollmentService và AdminService..	54
3.11. Triển khai rate limiting và audit log	54
3.11.1. Rate limiting	54
3.11.2. Audit log	55

3.12. Cấu hình bảo mật ở lớp ứng dụng và reverse proxy.....	55
3.12.1. CORS.....	55
3.12.2. Error handling.....	56
3.12.3. Security headers.....	56
3.13. Thiết kế giao diện frontend phục vụ demo	56
3.14. Docker hóa và kịch bản triển khai local	59
3.14.1. Cổng và truy cập.....	59
3.14.2. Biến môi trường chính.....	60
3.15. Dữ liệu mẫu phục vụ demo.....	60
3.16. Bảng tổng hợp một số kịch bản demo tấn công/phòng thủ	61
3.17. Kết quả đạt được.....	61
3.18. Hạn chế của hệ thống	62
3.19. Hướng phát triển.....	62
Chương 4. Thực nghiệm, demo và đánh giá hệ thống	65
4.1. Môi trường và điều kiện thực nghiệm	65
4.2. Dữ liệu mẫu và công cụ phục vụ thực nghiệm	66
4.2.1. Tài khoản mẫu	66
4.2.2. Công cụ sử dụng.....	66
4.2.3. Điều kiện chuẩn bị trước khi demo	67
4.3. Kịch bản 1: Kiểm chứng băm mật khẩu bằng bcrypt.....	67
4.3.1. Mục tiêu.....	67
4.3.2. Cách thực hiện	67
4.4.3. Kết quả quan sát được	70
4.4.4. Nhận xét.....	70
4.4. Kịch bản 2: Kiểm chứng JWT, Bearer Token, refresh token và logout.....	70
4.4.1. Mục tiêu.....	70

4.4.2. Các bước thực nghiệm chính.....	71
4.4.3. Kết quả quan sát được	74
4.4.4. Giá trị của kịch bản.....	74
4.5. Kịch bản 3: Kiểm chứng luồng quên mật khẩu và đặt lại mật khẩu	75
4.5.1. Mục tiêu.....	75
4.5.2. Cách thực hiện	75
4.5.3. Kết quả quan sát được	79
4.5.4. Nhận xét.....	79
4.6. Kịch bản 4: Kiểm chứng mã hóa AES-GCM cho dữ liệu nhạy cảm	79
4.6.1. Mục tiêu.....	79
4.6.2. Các dữ liệu được kiểm chứng.....	79
4.6.3. Cách thực hiện	80
4.6.4. Kết quả quan sát được	82
4.6.5. Đánh giá.....	82
4.7. Kịch bản 5: Chống BOLA/IDOR và kiểm soát quyền trên tài nguyên.....	83
4.7.1. Mục tiêu.....	83
4.7.2. Tình huống thực nghiệm chính.....	83
4.7.3. Kết quả quan sát được	85
4.7.4. Đánh giá.....	85
4.8. Kịch bản 6: Yêu cầu ghi danh và phê duyệt ghi danh	85
4.8.1. Mục tiêu.....	85
4.8.2. Cách thực hiện	86
4.8.3. Kết quả quan sát được	88
4.8.4. Đánh giá.....	88
4.9. Kịch bản 7: Rate limiting và ghi nhận hành vi bất thường	88
4.9.1. Mục tiêu.....	88

4.9.2. Cách thực hiện	89
4.9.3. Kết quả quan sát được	90
4.9.4. Đánh giá.....	90
4.10. Kịch bản 8: HTTPS/TLS, Swagger/OpenAPI và quét OWASP ZAP	90
4.10.1. Mục tiêu.....	90
4.10.2. Kết quả kiểm tra HTTPS/TLS	90
4.10.3. Kết quả kiểm tra Swagger/OpenAPI	91
4.10.4. Kết quả quét OWASP ZAP	92
4.10.5. Đánh giá.....	93
4.11. Đối chiếu với các tiêu chí chấp nhận của đề tài	93
4.12. Đánh giá tổng hợp về giá trị học thuật và thực hành.....	95
Chương 5. Kết luận và kiến nghị.....	96
5.1. Tóm tắt kết quả toàn đề tài	96
5.2. Đóng góp chính của đề tài	96
5.2.1. Đóng góp về mặt kỹ thuật	96
5.2.2. Đóng góp về mặt học thuật.....	97
5.2.3. Đóng góp về mặt trình bày đồ án	97
5.3. Ý nghĩa của đề tài đối với môn học.....	97
5.4. Hạn chế của đề tài.....	98
5.5. Hướng phát triển trong tương lai.....	99
Tài liệu tham khảo	100

Tóm tắt

Đề tài “Bảo mật hệ thống RESTful API cho dịch vụ khóa học online nhỏ” được thực hiện với mục tiêu xây dựng một hệ thống đủ gọn để phù hợp phạm vi môn Mật mã ứng dụng, nhưng vẫn thể hiện rõ các kỹ thuật bảo mật cốt lõi trong môi trường API hiện đại. Hệ thống được xây dựng với backend Spring Boot, Spring Security, frontend NextJS, cơ sở dữ liệu MySQL, reverse proxy Nginx và đóng gói bằng Docker Compose.

Hệ thống sử dụng bcrypt để băm mật khẩu, JWT để xác thực request, refresh token để quay vòng phiên đăng nhập, AES-GCM để mã hóa dữ liệu nhạy cảm trong cơ sở dữ liệu, HTTPS/TLS để bảo vệ dữ liệu khi truyền trên mạng, đồng thời triển khai rate limiting, audit log, RBAC, ownership check và luồng enrollment request -> admin approval -> lesson unlocked để giảm các rủi ro như brute-force, BOLA/IDOR, truy cập trái phép và lạm dụng logic nghiệp vụ.

Kết quả thực nghiệm cho thấy hệ thống đáp ứng tốt các mục tiêu đề ra. Mật khẩu không được lưu dưới dạng rõ, token bị sửa payload hoặc hết hạn bị từ chối, sinh viên không thể truy cập tài nguyên của sinh viên khác; yêu cầu ghi danh ở trạng thái PENDING chưa mở khóa bài học cho tới khi được quản trị viên duyệt thành ACTIVE; dữ liệu nhạy cảm trong database được lưu dưới dạng ciphertext; các hành vi bất thường được ghi nhận qua audit log; các endpoint nhạy cảm bị giới hạn tốc độ truy cập; toàn bộ hệ thống chạy ổn định và có thể kiểm thử bằng Swagger, Postman, OWASP ZAP.

Lời mở đầu

Trong bối cảnh hiện nay, phần lớn các hệ thống web hiện đại đều tách frontend và backend, giao tiếp với nhau chủ yếu qua API. Điều đó khiến RESTful API trở thành bề mặt tấn công rất quan trọng. Khi API không được bảo vệ đúng cách, kẻ tấn công có thể bỏ qua giao diện người dùng và tác động trực tiếp vào request HTTP để brute-force mật khẩu, giả mạo token, thay đổi định danh tài nguyên hoặc gọi trái phép các endpoint nhạy cảm.

Với sinh viên ngành An toàn thông tin, việc chỉ học lý thuyết về mật mã là chưa đủ. Điều quan trọng hơn là phải hiểu mỗi kỹ thuật mật mã được dùng để bảo vệ loại dữ liệu nào, đặt ở lớp nào trong hệ thống và chứng minh được hiệu quả của nó thông qua mã nguồn và thực nghiệm. Từ yêu cầu đó, nhóm lựa chọn xây dựng một hệ thống khóa học online nhỏ theo hướng bảo mật API, trong đó mọi thành phần đều được thu gọn vừa đủ để dễ cài đặt, dễ demo và dễ giải thích trước giảng viên.

Điểm đặc biệt của đề tài là không sa đà vào việc làm một sản phẩm công nghệ phần mềm hoàn chỉnh với quá nhiều nghiệp vụ. Thay vào đó, nhóm tập trung vào các nội dung có tính “Mật mã ứng dụng” rõ rệt như băm mật khẩu bằng bcrypt, mã hóa dữ liệu nhạy cảm bằng AES-GCM, xác thực request bằng JWT, bảo vệ kênh truyền bằng TLS, kiểm tra quyền sở hữu tài nguyên để chống BOLA/IDOR, giới hạn tần suất truy cập, ghi nhận audit log và dùng trạng thái ghi danh PENDING/ACTIVE để gắn bảo mật với logic nghiệp vụ mở khóa bài học.

Báo cáo này được xây dựng với định hướng vừa mang tính học thuật, vừa mang tính thực hành. Ngoài phần cơ sở lý thuyết, báo cáo còn trình bày cách thiết kế hệ thống, tổ chức mã nguồn, mô hình dữ liệu, quy trình kiểm thử và các kịch bản demo tấn công/phòng thủ. Mục tiêu cuối cùng là chứng minh rằng các kỹ thuật mật mã không chỉ tồn tại ở mức khái niệm, mà có thể được tích hợp đúng cách vào một hệ thống chạy thật, có thể quan sát, đo kiểm và đánh giá.

Chương 1. Tổng quan đề tài

1.1. Đặt vấn đề

Trong các hệ thống phần mềm hiện đại, RESTful API đóng vai trò là lớp giao tiếp trung tâm giữa frontend, backend, ứng dụng di động và các dịch vụ bên thứ ba. Khi API trở thành “cửa ngõ” của hệ thống, các yêu cầu về an toàn thông tin không còn là phần phụ mà trở thành điều kiện bắt buộc. Nếu lớp API không được thiết kế và bảo vệ đúng cách, kẻ tấn công có thể:

- Đăng nhập trái phép bằng tài khoản đánh cắp hoặc brute-force mật khẩu.
- Sửa token hoặc giả mạo thông tin trong token để leo thang đặc quyền.
- Thay đổi định danh tài nguyên trên URL để xem dữ liệu của người khác.
- Đọc được dữ liệu nhạy cảm nếu dữ liệu trong cơ sở dữ liệu bị lộ ở dạng rõ.
- Lợi dụng logic ghi danh để truy cập bài học khi yêu cầu học chưa được phê duyệt.
- Gây nghẽn dịch vụ bằng cách gửi số lượng lớn request tới các endpoint nhạy cảm.

Trong bối cảnh đó, nhóm lựa chọn xây dựng một hệ thống khóa học online nhỏ theo định hướng “Mật mã ứng dụng và bảo mật API”. Trọng tâm của đề tài không phải là xây dựng một nền tảng học trực tuyến hoàn chỉnh như sản phẩm thương mại, mà là hiện thực hóa các kỹ thuật mật mã và cơ chế bảo mật.

1.2. Bối cảnh thực tiễn của bài toán

Hệ thống được đặt trong bối cảnh một doanh nghiệp nhỏ cung cấp các khóa học online. Do quy mô nhỏ, hệ thống không cần có đầy đủ các tính năng phức tạp như recommendation engine, livestream hay phân tích dữ liệu lớn. Tuy nhiên, những thành phần cốt lõi liên quan đến bảo mật vẫn được bảo vệ chặt chẽ.

1.2.1. Ngõ cảnh hệ thống và ranh giới tin cậy

Về mặt an toàn thông tin, hệ thống được nhìn dưới góc độ các vùng tin cậy sau:

- **Vùng công cộng:** trình duyệt người dùng, Postman, OWASP ZAP và các request HTTP xuất phát từ bên ngoài.
- **Vùng biên (edge):** Nginx làm reverse proxy, điểm kết thúc TLS và điểm áp chính sách truy cập theo host/path.
- **Vùng ứng dụng:** NextJS frontend và Spring Boot backend; trong đó backend là nơi thực thi các kiểm tra bảo mật bắt buộc.
- **Vùng dữ liệu:** MySQL/H2, nơi lưu tài khoản, refresh token hash, ciphertext AES-GCM, enrollment, certificate và audit log.
- **Vùng quản trị nội bộ:** các bề mặt chỉ nên dùng khi vận hành trên máy chủ hoặc môi trường local như Swagger UI, truy cập database để kiểm chứng dữ liệu và một số công cụ demo/kiểm thử.

Từ cách phân vùng này có thể xác định các ranh giới tin cậy quan trọng:

1. Ranh giới giữa Internet và reverse proxy TLS.
2. Ranh giới giữa reverse proxy và backend ứng dụng.
3. Ranh giới giữa backend và database.
4. Ranh giới giữa người dùng thông thường với các tài nguyên thuộc về người dùng khác.
5. Ranh giới giữa client bình thường với các bề mặt quản trị/kiểm thử nội bộ.

1.2.2. Tài sản cần bảo vệ

Các tài sản quan trọng của hệ thống gồm:

Tài sản	Ý nghĩa	Rủi ro nếu không bảo vệ
Nội dung bài học	Tài nguyên có giá trị thương mại	Người chưa mua vẫn xem được nếu thiếu kiểm soát truy cập
Enrollment và certificate	Chứng minh quyền học và kết quả học tập	Bị truy cập chéo giữa các sinh viên hoặc bị cấp sai
Tài khoản người dùng	Dùng để xác thực và phân quyền	Bị chiếm đoạt, truy cập trái phép
Mật khẩu	Bí mật đăng nhập	Bị lộ nếu lưu plaintext hoặc hash yếu
JWT access token	Chứng thực cho các request API	Bị sửa payload, giả mạo chữ ký, dùng token hết hạn
Refresh token	Cấp lại access token	Bị tái sử dụng nếu không hash hoặc không revoke
Dữ liệu cá nhân	Số điện thoại, billingAddress trong hồ sơ người học	Bị lộ nếu database rò rỉ
Trạng thái ghi danh và quy trình phê duyệt	Quyết định thời điểm sinh viên được mở khóa khóa học	Bị lạm dụng nếu PENDING vẫn truy cập như ACTIVE hoặc nếu API duyệt không tách quyền

Audit log	Dữ liệu giám sát an ninh	Mất dấu vết nếu hệ thống không ghi log
-----------	--------------------------	--

1.2.3. Tác nhân đe dọa và kịch bản tấn công chính

Các tác nhân và kịch bản với bài toán API của một dịch vụ nhỏ:

Tác nhân đe dọa	Năng lực giả định	Mục tiêu chính	Kịch bản cần phòng vệ
Kẻ tấn công ẩn danh từ Internet	Có thể gửi request thủ công, dùng Postman/ZAP, brute-force, sửa header/payload	Chiếm tài khoản, dò endpoint, spam API	brute-force login, token tampering, gọi sai định dạng Bearer, spam request
Sinh viên hợp lệ nhưng tò mò hoặc có chủ đích xấu	Có tài khoản thật và token hợp lệ của chính mình	Xem dữ liệu không thuộc quyền hoặc tự mở khóa tài nguyên	đổi userId , enrollmentId , certificateId , lessonId ; thử gọi API admin; thử dùng enrollment PENDING như ACTIVE
Tài khoản quản trị hoặc quy trình phê duyệt bị lạm dụng	Có thể gọi API quản trị hợp lệ hoặc thao tác nhằm	Duyệt nhằm ghi danh, cấp quyền học sai đối tượng	gọi approve sai enrollment, thêm nhằm học viên vào khóa học, không tách rõ

			PENDING và ACTIVE
Đối tượng có được bản sao database hoặc backup	Có thể đọc dữ liệu lưu trữ nhưng không có secret runtime	Lấy password, refresh token, dữ liệu cá nhân	khai thác password plaintext, refresh token plaintext, phone/address/certifica te code ở dạng rõ
Người vận hành cấu hình sai	Có quyền cấu hình nhưng không kiểm soát chặt bề mặt exposed	Làm lộ bề mặt quản trị hoặc secret	đề lộ Swagger public, cấu hình CORS quá rộng, hard-code secret trong source

1.2.4. Đánh giá rủi ro trọng yếu

Từ ngữ cảnh và tác nhân đe dọa ở trên, nhóm xác định một số rủi ro bảo mật trọng yếu cần ưu tiên xử lý trong đồ án:

Rủi ro	Khả năng xảy ra	Tác động	Mức ưu tiên	Hướng xử lý chính
Brute-force đăng nhập và spam API	Cao	Trung bình đến cao	Cao	bcrypt , rate limiting , audit log , response không lộ quá nhiều chi tiết
JWT bị sửa payload, dùng token sai định	Cao	Cao	Cao	kiểm tra chữ ký JWT, exp , Bearer format, từ chối token sai

dạng hoặc token hết hạn				
BOLA/IDOR ở certificate, enrollment, lesson, profile	Cao	Cao	Rất cao	RBAC kết hợp ownership check ở backend, ghi ACCESS_DENIED
Lộ dữ liệu khi database bị rò rỉ	Trung bình	Cao	Cao	bcrypt cho password, hash refresh token, AES-GCM cho dữ liệu nhạy cảm
Tách quyền không chặt giữa STUDENT và ADMIN , hoặc dùng sai trạng thái PENDING/ACTIVE	Trung bình	Cao	Cao	RBAC , endpoint duyệt chỉ cho ADMIN , kiểm tra enrollment ACTIVE trước khi mở lesson hoặc certificate
Bề mặt quản trị hoặc kiểm thử bị lộ công khai	Trung bình	Trung bình đến cao	Trung bình	quản lý rõ việc mở Swagger/OpenAPI qua Nginx và tunnel public, chỉ bật public demo khi cần, tách rõ bề mặt admin, cấu hình CORS rõ ràng

1.3. Lý do chọn đề tài

Đề tài được lựa chọn vì ba lý do chính.

Thứ nhất, đề tài bám sát định hướng của học phần Mật mã ứng dụng. Hệ thống tích hợp các cơ chế bảo mật cốt lõi như bcrypt để bảo vệ mật khẩu, AES-GCM để mã hóa dữ liệu nhạy cảm, JWT sử dụng khóa bí mật đối xứng để xác thực và HTTPS/TLS để bảo vệ dữ liệu trong quá trình truyền. Việc triển khai các cơ chế này trong một hệ thống thực tế giúp người học hiểu rõ mục đích sử dụng, quy trình hoạt động, phạm vi áp dụng cũng như ưu, nhược điểm của từng kỹ thuật, thay vì chỉ tiếp cận ở mức lý thuyết.

Thứ hai, bảo mật API là một chủ đề có tính thực tiễn và được ứng dụng rộng rãi trong các hệ thống hiện đại. Phần lớn các ứng dụng hiện nay được phát triển theo mô hình frontend và backend tách biệt, giao tiếp thông qua REST API. Điều này làm gia tăng nguy cơ xuất hiện các lỗ hổng như Broken Authentication, Broken Object Level Authorization (BOLA/IDOR), Excessive Data Exposure và Security Misconfiguration. Vì vậy, việc xây dựng một hệ thống tập trung vào bảo vệ API giúp nội dung đồ án phản ánh đúng những vấn đề an toàn thông tin thường gặp trong thực tế.

Thứ ba, đề tài phù hợp với khả năng triển khai của sinh viên nhưng vẫn đảm bảo chiều sâu về mặt kỹ thuật. Hệ thống được thiết kế với phạm vi nghiệp vụ vừa đủ để hoàn thành trong thời gian thực hiện đồ án, đồng thời tích hợp đầy đủ các cơ chế bảo mật, cho phép xây dựng các kịch bản minh họa tấn công và phòng thủ, kiểm thử bằng Postman và OWASP ZAP, cũng như triển khai toàn bộ hệ thống bằng Docker để tăng tính thực tiễn và khả năng tái hiện môi trường vận hành.

1.4. Mục tiêu của đề tài

1.4.1. Mục tiêu tổng quát

Xây dựng một hệ thống RESTful API cho dịch vụ khóa học online nhỏ và tích hợp các kỹ thuật mật mã ứng dụng, trong đó nhấn mạnh việc bảo vệ xác thực, bảo vệ dữ liệu

nhạy cảm, bảo vệ quyền truy cập tài nguyên, bảo vệ logic ghi danh, bảo vệ đường truyền và hỗ trợ kiểm thử bảo mật.

1.4.2. Mục tiêu cụ thể

Các mục tiêu cụ thể của đề tài gồm:

- Xây dựng backend bằng Spring Boot và Spring Security.
- Xây dựng frontend tối giản bằng NextJS.
- Sử dụng JWT làm access token theo mô hình Authorization.
- Cung cấp refresh token, lưu trong database dưới dạng hash để giảm rủi ro lộ token.
- Xây dựng luồng quên mật khẩu và đặt lại mật khẩu để bảo đảm vòng đời xác thực tài khoản đầy đủ.
- Băm mật khẩu bằng BCryptPasswordEncoder, không lưu plaintext password.
- Mã hóa dữ liệu nhạy cảm bằng AES-GCM.
- Hiện thực luồng STUDENT gửi yêu cầu ghi danh và ADMIN phê duyệt để kiểm soát quyền truy cập bài học.
- Tổ chức quyền tối giản, tập trung vào hai vai trò demo chính là STUDENT và ADMIN.
- Chặn BOLA/IDOR bằng kiểm tra ownership ở phía backend.
- Giới hạn tần suất truy cập với các endpoint nhạy cảm để chống brute-force và spam.
- Ghi audit log cho các sự kiện bảo mật quan trọng.
- Cấu hình HTTPS/TLS cục bộ qua Nginx.
- Tích hợp Swagger/OpenAPI, Postman, OWASP ZAP để kiểm thử.
- Đóng gói hệ thống bằng Docker để chạy nhất quán trên môi trường local.

1.4.3. Mục tiêu bảo mật

Xuất phát từ ngữ cảnh hệ thống và các rủi ro đã nêu, đề án đặt ra các mục tiêu bảo mật cụ thể sau:

Mục tiêu bảo mật	Nội dung cần đạt	Cơ chế áp dụng chính
Bảo vệ danh tính và phiên đăng nhập	Chỉ người dùng hợp lệ mới đăng nhập được, token sai hoặc hết hạn phải bị từ chối	bcrypt , JWT , refresh token hash, logout/revoke, forgot/reset password
Bảo vệ quyền truy cập tài nguyên	Sinh viên không được xem dữ liệu của sinh viên khác, admin và student bị tách quyền rõ ràng	Spring Security , RBAC , ownership check , 403 Forbidden
Bảo vệ dữ liệu nhạy cảm khi lưu trữ	Nếu database bị lộ, dữ liệu quan trọng không ở dạng plaintext	AES-GCM , hash password, hash refresh token
Bảo vệ logic mở khóa khóa học	Sinh viên chỉ xem được nội dung bài học khi ghi danh đã được duyệt hợp lệ	trạng thái PENDING/ACTIVE , endpoint duyệt chỉ cho ADMIN , kiểm tra enrollment ACTIVE trước khi trả lesson/certificate
Bảo vệ kênh truyền và bề mặt exposed	Dữ liệu đi qua HTTPS, việc mở bề mặt kiểm thử ra public phải được kiểm soát theo nhu cầu demo	HTTPS/TLS , Nginx , quản lý host/domain public, ưu tiên localhost , chỉ bật tunnel khi cần

Bảo vệ khả năng giám sát và phản ứng	Hệ thống phải ghi nhận được hành vi bất thường và giảm thiểu lạm dụng tài nguyên	audit log , rate limiting , cảnh báo qua status code phù hợp
--------------------------------------	--	--

1.5. Câu hỏi nghiên cứu và định hướng giải quyết

Đề tài tập trung trả lời các câu hỏi sau:

1. Làm thế nào để sử dụng JWT trong RESTful API sao cho token bị sửa payload hoặc sai chữ ký sẽ bị từ chối?
2. Làm thế nào để lưu mật khẩu người dùng an toàn bằng cơ chế băm một chiều thay vì mã hóa?
3. Làm thế nào để bảo vệ dữ liệu nhạy cảm trong cơ sở dữ liệu ngay cả khi tầng lưu trữ bị lộ?
4. Làm thế nào để phân biệt rõ xác thực, phân quyền theo vai trò và kiểm tra quyền sở hữu tài nguyên?
5. Làm thế nào để chứng minh hệ thống đã xử lý được các lỗi phổ biến trong OWASP API Security Top 10?
6. Làm thế nào để xây dựng một luồng ghi danh đủ nhỏ nhưng vẫn cho thấy rõ tác động của role, trạng thái nghiệp vụ và ownership lên quyền truy cập tài nguyên?

Định hướng giải quyết của nhóm là không dàn trải quá nhiều nghiệp vụ, mà ưu tiên làm rõ từng lớp bảo vệ bằng các luồng xử lý đủ gọn, dễ đọc, dễ demo và dễ giải thích.

1.6. Đối tượng và phạm vi nghiên cứu

1.6.1. Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là các cơ chế bảo mật trong hệ thống API web hiện đại, đặc biệt là:

- Cơ chế xác thực bằng token.
- Cơ chế phân quyền và kiểm soát truy cập tài nguyên.
- Kỹ thuật băm mật khẩu.
- Kỹ thuật mã hóa đối xứng dữ liệu khi lưu trữ.
- Kỹ thuật bảo vệ tính toàn vẹn token dựa trên secret đối xứng.
- Kiểm thử bảo mật cho API.

1.6.2. Phạm vi triển khai

Nội dung	Trong phạm vi	Ngoài phạm vi
Xác thực người dùng	Đăng ký, đăng nhập, access token, refresh token, logout	Social login, SSO, OAuth2 với Google/Facebook
Nghiệp vụ khóa học	Danh sách khóa học, chi tiết khóa học, lesson, enrollment, certificate	LMS đầy đủ, video streaming, bài tập nâng cao
Bảo mật dữ liệu	bcrypt, AES-GCM, JWT signing, TLS	KMS, HSM, key rotation production-grade
Kiểm thử	Swagger, Postman, OWASP ZAP, integration test	Pentest chuyên sâu, bug bounty, red team
Triển khai	Docker Compose local, Nginx HTTPS	Kubernetes, CI/CD production, auto scaling

1.7. Phương pháp thực hiện

Nhóm triển khai đề tài theo phương pháp kết hợp giữa nghiên cứu lý thuyết và thực nghiệm hệ thống.

1.7.1. Nghiên cứu lý thuyết

Nhóm nghiên cứu các nội dung nền tảng gồm:

- RESTful API và các phương thức HTTP.
- JWT, Bearer Token, refresh token.
- bcrypt và nguyên tắc băm mật khẩu.
- AES-GCM và tính bảo mật kèm toàn vẹn.
- RBAC, ownership check và trạng thái nghiệp vụ trong kiểm soát truy cập.
- HTTPS/TLS và reverse proxy.
- OWASP API Security Top 10.

1.7.2. Lựa chọn nền tảng kỹ thuật và xác định phạm vi xây dựng

Sau khi xác định mục tiêu bảo mật của đề tài, nhóm lựa chọn nền tảng kỹ thuật phù hợp để xây dựng hệ thống từ đầu:

- Dùng Spring Boot và Spring Security để xây dựng backend RESTful API và cường chế xác thực, phân quyền ở server-side.
- Dùng NextJS để làm giao diện demo các luồng đăng ký, đăng nhập, quên mật khẩu, xem khóa học, gửi yêu cầu ghi danh và truy cập tài nguyên protected.
- Dùng MySQL/H2, Nginx, Docker Compose, Swagger, Postman và OWASP ZAP để phục vụ lưu trữ, reverse proxy, chạy local và kiểm thử bảo mật.
- Xác định phạm vi nghiệp vụ ở mức vừa đủ cho đề án: User, Auth, Course, Lesson, Enrollment, Certificate, Admin Enrollment Review.
- Xác định rõ trọng tâm bảo mật cần hiện thực: JWT, refresh token, bcrypt, AES-GCM, RBAC, ownership check, audit log, rate limiting, HTTPS/TLS.

1.7.3. Thiết kế và hiện thực hệ thống

Sau khi chốt nền tảng và phạm vi xây dựng, nhóm tiến hành thiết kế và hiện thực theo từng lớp:

- Thiết kế mô hình dữ liệu, vai trò người dùng, use case và các luồng bảo mật chính của hệ thống.
- Thiết kế API cho đăng ký, đăng nhập, refresh token, forgot/reset password, gửi yêu cầu ghi danh, phê duyệt ghi danh và truy cập tài nguyên protected.
- Hiện thực backend theo các lớp entity, repository, service, controller, security filter và các service bảo mật cần thiết.
- Hiện thực frontend demo bám theo đúng các luồng nghiệp vụ và luồng kiểm soát bảo mật của backend, đặc biệt là trạng thái PENDING và ACTIVE của enrollment.
- Chuẩn bị tài liệu, script và dữ liệu mẫu để phục vụ demo, kiểm thử và viết báo cáo.

1.7.4. Kiểm thử và đánh giá

Nhóm đánh giá hệ thống qua nhiều lớp kiểm thử:

- Test backend bằng Maven.
- Kiểm thử thủ công bằng Swagger.
- Kiểm thử tình huống bằng Postman.
- Chạy script demo tự động.
- Quét baseline bằng OWASP ZAP.

1.8. Công cụ và môi trường sử dụng

Thành phần	Công nghệ sử dụng	Vai trò
Backend	Java, Spring Boot, Spring Security	Xây dựng REST API và xử lý bảo mật

Frontend	NextJS	Giao diện demo luồng bảo mật
Database	MySQL, H2	Lưu trữ dữ liệu hệ thống và test local
API Docs	Springdoc OpenAPI / Swagger UI	Tài liệu và thử nghiệm API
Reverse proxy	Nginx	TLS termination, HTTP to HTTPS redirect
Kiểm thử	Postman, OWASP ZAP	Kiểm thử thủ công và quét bảo mật
Triển khai	Docker, Docker Compose	Chạy đồng bộ toàn stack
Hệ điều hành thực nghiệm	Windows + Docker Desktop	Môi trường demo local

1.9. Kết quả mong đợi của đề tài

Sau khi hoàn thành, hệ thống cần đạt các kết quả sau:

- Chạy được frontend và backend.
- Người dùng có thể đăng ký, đăng nhập, làm mới token, đăng xuất, quên mật khẩu và đặt lại mật khẩu.
- Password lưu trong database ở dạng bcrypt hash.
- Các trường nhạy cảm lưu ở dạng ciphertext AES-GCM.
- Sinh viên không thể xem dữ liệu của sinh viên khác bằng cách sửa ID trên URL.
- Sinh viên chỉ xem được lesson sau khi enrollment được admin duyệt hợp lệ.

- Hệ thống có audit log và rate limiting.
- Swagger có thể test API protected bằng Bearer JWT.
- Có bộ tài liệu và artifact phục vụ viết báo cáo, demo bằng Postman và đánh giá bằng OWASP ZAP.

1.10. Ý nghĩa khoa học và thực tiễn

Về mặt khoa học, đề tài thể hiện được sự khác nhau giữa các cơ chế mật mã:

- bcrypt dùng cho băm mật khẩu một chiều.
- AES-GCM dùng cho mã hóa dữ liệu cần giải mã lại.
- HMAC dùng cho việc xác minh tính toàn vẹn của JWT ký đối xứng.
- JWT dùng để đóng gói claims và xác minh bằng chữ ký số đối xứng.

Về mặt thực tiễn, đề tài có thể được dùng như một mô hình mẫu cho các hệ thống API quy mô nhỏ cần:

- Bảo vệ xác thực người dùng.
- Bảo vệ dữ liệu cá nhân.
- Bảo vệ tài nguyên số có giá trị.
- Ghi nhận và giám sát hành vi bất thường.

1.11. Bố cục báo cáo

- Chương 1 trình bày tổng quan đề tài, bối cảnh, mục tiêu, phạm vi và phương pháp thực hiện.
- Chương 2 trình bày cơ sở lý thuyết và các công nghệ bảo mật làm nền tảng cho đề tài.
- Chương 3 trình bày quá trình phân tích, thiết kế, hiện thực và kiểm thử hệ thống.
- Chương 4 trình bày quá trình thực nghiệm, demo tấn công/phòng thủ và đánh giá mức độ đáp ứng các tiêu chí bảo mật.
- Chương 5 tổng kết kết quả đạt được, nêu hạn chế, hướng phát triển và kết luận chung của đề tài.

Chương 2. Cơ sở lý thuyết và công nghệ sử dụng

2.1. Một số khái niệm nền tảng trong an toàn thông tin

Khi bảo vệ một hệ thống API, cần xuất phát từ ba thuộc tính cơ bản của an toàn thông tin:

- **Tính bí mật:** dữ liệu chỉ được xem bởi đối tượng hợp lệ.
- **Tính toàn vẹn:** dữ liệu không bị sửa đổi trái phép.
- **Tính sẵn sàng:** hệ thống duy trì khả năng phục vụ hợp lệ.

Đề tài này đồng thời cũng gắn với mô hình AAA trong bảo mật:

- Authentication: xác thực người dùng.
- Authorization: phân quyền truy cập.
- Accounting/Auditing: ghi nhận và giám sát hành động.

Mỗi kỹ thuật được dùng trong đồ án sẽ phục vụ một hoặc nhiều mục tiêu ở trên.

2.2. Phân biệt các cơ chế mật mã sử dụng trong đề tài

Để tránh nhầm lẫn giữa băm, mã hóa và xác thực thông điệp, Bảng 2.1 tóm tắt vai trò của từng cơ chế.

Cơ chế	Bản chất	Có giải mã ngược không	Mục đích trong đề tài
bcrypt	Băm một chiều có salt	Không	Lưu mật khẩu
AES-GCM	Mã hóa đối xứng	Có	Bảo vệ dữ liệu nhạy cảm trong database
HMAC-SHA256	Mã xác thực thông điệp	Không phải cơ chế giải mã	Bảo vệ tính toàn vẹn của JWT ký đối xứng

JWT ký HMAC	Token có chữ ký	Không mã hóa payload	Xác thực request API
----------------	-----------------	-------------------------	----------------------

Điểm quan trọng là không sử dụng sai mục đích:

- Không dùng AES để “mã hóa mật khẩu”.
- Không dùng bcrypt để lưu dữ liệu cần đọc lại.
- Không coi JWT là dữ liệu bí mật tuyệt đối vì payload có thể được giải mã base64.

2.3. Tổng quan về RESTful API

RESTful API là phong cách thiết kế dịch vụ web trong đó tài nguyên được biểu diễn qua URL và thao tác bằng HTTP method. Một số đặc điểm chính:

- GET: đọc tài nguyên.
- POST: tạo tài nguyên hoặc kích hoạt hành động.
- PUT: cập nhật tài nguyên.
- DELETE: xóa tài nguyên.

Ưu điểm của RESTful API là:

- Đơn giản, dễ tích hợp.
- Phù hợp với frontend web, mobile và bên thứ ba.
- Dễ mô tả, tài liệu hóa và kiểm thử.

Tuy nhiên, chính vì API mở ra bề mặt giao tiếp trực tiếp nên đây cũng là bề mặt tấn công phổ biến. Trong nhiều trường hợp, kẻ tấn công không cần truy cập giao diện người dùng mà có thể trực tiếp sửa request HTTP để thử token, đổi ID tài nguyên hoặc gửi lượng lớn request bất thường.

2.4. Xác thực và phân quyền trong hệ thống API

Trong một hệ thống API, cần phân biệt rõ:

- **Xác thực:** người dùng là ai?
- **Phân quyền:** người dùng được phép làm gì?
- **Kiểm tra ownership:** người dùng có quyền với đúng đối tượng dữ liệu đó không?

Ví dụ:

- Một request có JWT hợp lệ nghĩa là người dùng đã được xác thực.
- Tuy nhiên, cùng là người dùng đã xác thực, STUDENT không thể xem audit log.
- Ngay cả khi đều là STUDENT, sinh viên A cũng không được xem certificate của sinh viên B.

2.5. JWT và Bearer Token

2.5.1. Cấu trúc JWT

JWT gồm ba phần:

1. Header
2. Payload
3. Signature

Hai phần đầu thường được mã hóa Base64Url, còn phần thứ ba là chữ ký dùng secret key. Trong đồ án, access token chứa các claim quan trọng:

- sub: mã người dùng.
- email: email người dùng.
- role: vai trò.
- scope: tập quyền.
- token_type: loại token.
- iat: thời điểm phát hành.
- exp: thời điểm hết hạn.

2.5.2. Vai trò của Bearer Token

Sau khi đăng nhập thành công, client nhận được access token và gửi trong header:

```
Authorization: Bearer <access_token>
```

Backend sẽ đọc token, kiểm tra chữ ký, kiểm tra thời hạn, kiểm tra role và sau đó gắn danh tính người dùng vào security context.

2.5.3. Các rủi ro cần kiểm soát

JWT chỉ an toàn khi backend thực hiện đầy đủ các bước kiểm tra:

- Từ chối token sai định dạng.
- Từ chối token không có chữ ký hợp lệ.
- Từ chối token hết hạn.
- Từ chối token thiếu claim bắt buộc.
- Từ chối token có token_type không đúng.

Trong dự án này, secret ký JWT không được hard-code mà lấy từ biến môi trường JWT_SECRET. Access token có thời gian sống ngắn mặc định 15 phút, còn refresh token mặc định 7 ngày.

2.5.4. Refresh token

Refresh token trong dự án được sinh ngẫu nhiên, trả về cho client nhưng lưu trong database ở dạng SHA-256 hash. Cách làm này giúp:

- Không lưu token gốc trong database.
- Có thể revoke token khi logout hoặc khi cấp token mới.
- Giảm rủi ro nếu bảng refresh_tokens bị truy cập trái phép.

2.6. Băm mật khẩu bằng bcrypt

bcrypt là thuật toán băm một chiều được thiết kế cho mật khẩu. Khác với các hàm băm nhanh như SHA-256, bcrypt có cơ chế salt và cost factor để làm chậm quá trình băm khóa.

Các đặc điểm chính của bcrypt:

- Là băm một chiều, không thể “giải mã”.
- Tự sinh salt cho từng mật khẩu.
- Kết quả cùng một mật khẩu ở hai lần băm có thể khác nhau.
- Thích hợp để chống brute-force và rainbow table tốt hơn so với băm nhanh.

Trong dự án:

- Khi đăng ký, backend gọi `passwordEncoder.encode(rawPassword)`.
- Khi đăng nhập, backend gọi `passwordEncoder.matches(rawPassword, passwordHash)`.
- API response không bao giờ trả về `passwordHash`.
- Log hệ thống không in ra mật khẩu gốc.

2.7. Mã hóa dữ liệu nhạy cảm bằng AES-GCM

2.7.1. Vì sao cần mã hóa dữ liệu lưu trữ

Không phải dữ liệu nào cũng phù hợp để băm. Với những trường cần hiển thị lại cho người dùng hợp lệ, hệ thống phải có khả năng giải mã. Ví dụ:

- `phoneNumber`
- `billingAddress`
- `certificateCode`

Nếu lưu những trường này ở dạng plaintext, khi database bị lộ, dữ liệu cá nhân và dữ liệu nghiệp vụ cũng bị lộ trực tiếp.

2.7.2. Đặc điểm của AES-GCM

AES-GCM là chế độ mã hóa đối xứng có xác thực, mang lại hai thuộc tính cùng lúc:

- Mã hóa để đảm bảo tính bí mật.
- Authentication tag để phát hiện dữ liệu bị chỉnh sửa.

Trong dự án, `EncryptionService`:

- Dẫn xuất khóa từ biến môi trường ENCRYPTION_KEY bằng SHA-256.
- Sinh IV ngẫu nhiên với kích thước 12 byte.
- Dùng tag dài 16 byte.
- Lưu kết quả theo định dạng:

```
Base64(iv):Base64(ciphertext):Base64(tag)
```

2.7.3. AAD trong AES-GCM

Một điểm quan trọng của dự án là ngoài ciphertext, hệ thống còn gắn AAD (Additional Authenticated Data) theo ngữ cảnh:

```
contextType | contextReference | fieldName
```

Ví dụ:

- user | email | phoneNumber
- user | email | billingAddress
- certificate | studentId|courseId | certificateCode

Cách làm này giúp ràng buộc dữ liệu với đúng ngữ cảnh nghiệp vụ. Nếu một ciphertext bị trao sang bản ghi khác hoặc đổi field, quá trình giải mã sẽ thất bại.

2.8. HMAC-SHA256

HMAC là cơ chế xác thực thông điệp sử dụng secret key kết hợp với hàm băm mật mã. Trong đề tài hiện tại, vai trò HMAC xuất hiện trong phần chữ ký của JWT access token thay vì một luồng callback tách riêng.

Về nguyên lý, JWT ký đối xứng có thể được mô tả khái quát như sau:

```
signature = HMAC_SHA256(base64url(header) + "." + base64url(payload), secret)
```

Ý nghĩa:

- header: mô tả loại token và thuật toán ký.
- payload: chứa các claim như sub, email, role, scope, iat, exp.

- secret: khóa bí mật chỉ backend nắm để ký và xác minh token.

Ưu điểm của HMAC:

- Bên nhận có thể phát hiện payload bị sửa đổi.
- Việc xác minh diễn ra nhanh và phù hợp với hệ thống API dùng secret đối xứng.
- Kết hợp tốt với kiểm tra exp, token_type, role và scope ở backend.

2.9. HTTPS/TLS

HTTPS là HTTP chạy trên TLS. Trong đề tài, TLS được dùng để bảo vệ:

- Thông tin đăng nhập khi người dùng gửi email và mật khẩu.
- JWT access token khi frontend gọi API.
- Dữ liệu nhạy cảm trao đổi giữa các thành phần của hệ thống.

Nginx được dùng làm reverse proxy để:

- Lắng nghe cổng 80 và 443.
- Chuyển toàn bộ HTTP sang HTTPS bằng 301.
- Phục vụ frontend và proxy request API tới backend.
- Thêm một số security header cơ bản như:
- Strict-Transport-Security
- Content-Security-Policy
- X-Content-Type-Options
- X-Frame-Options
- Referrer-Policy
- Permissions-Policy

2.10. RBAC, ownership check và chống BOLA/IDOR

RBAC là mô hình phân quyền theo vai trò. Tuy nhiên, chỉ dùng RBAC là chưa đủ đối với các tài nguyên cá nhân. Đề tài kết hợp hai lớp:

- **Role-based access control:** giới hạn theo vai trò.

- **Ownership check:** giới hạn theo quyền sở hữu tài nguyên.

Ví dụ:

- ADMIN được xem audit log.
- STUDENT chỉ được xem enrollment, certificate và profile của chính mình.
- Hệ thống chỉ trình diễn hai vai trò chính là STUDENT và ADMIN để giữ đúng trọng tâm mật mã ứng dụng và bảo mật API.

Lỗi BOLA/IDOR xảy ra khi backend chỉ dựa vào ID trên URL mà không kiểm tra ID đó có thuộc về người dùng hiện tại hay không. Đây là một trong những nội dung được nhấn mạnh nhất trong dự án.

2.11. Excessive Data Exposure và DTO response

Một API có thể xác thực đúng nhưng vẫn không an toàn nếu trả ra quá nhiều thông tin.

Ví dụ:

- Trả cả passwordHash.
- Trả trực tiếp ciphertext nội bộ.
- Trả các field chỉ dành cho admin.

Để tránh lỗi này, hệ thống sử dụng DTO cho response. Ví dụ:

- UserResponse chỉ chứa id, fullName, email, role, phoneNumber, billingAddress, createdAt.
- CourseDetailResponse chỉ trả thông tin cần thiết của khóa học và preview lesson.
- LessonDetailResponse chỉ trả content khi người dùng đủ quyền.

2.12. Rate limiting và audit logging

2.12.1. Rate limiting

Rate limiting là cơ chế giới hạn số lượng request trong một khoảng thời gian. Trong đề tài, rate limit được áp dụng cho:

- POST /api/auth/login
- POST /api/auth/register
- POST /api/auth/forgot-password
- GET /api/admin/**

Mục tiêu là:

- Giảm brute-force mật khẩu.
- Giảm spam đăng ký.
- Giảm spam yêu cầu quên mật khẩu.
- Giảm việc lạm dụng API quản trị.

2.12.2. Audit log

Audit log là lớp ghi nhận các hành vi quan trọng phục vụ giám sát và truy vết. Dự án ghi log cho:

- Đăng ký thành công.
- Đăng nhập thành công.
- Đăng nhập thất bại.
- Token bị từ chối.
- Truy cập bị chặn 403.
- Yêu cầu ghi danh được tạo hoặc được phê duyệt.
- Rate limit bị vượt ngưỡng.
- Admin xem audit log hoặc quản lý enrollment.
- Vượt ngưỡng rate limit.

2.13. Swagger/OpenAPI

Swagger/OpenAPI là công cụ tài liệu hóa và kiểm thử API rất phù hợp với đồ án. Hệ thống hiện tại:

- Công bố Swagger UI tại <https://localhost/swagger-ui.html>.

- Công bố OpenAPI JSON tại <https://localhost/v3/api-docs>.
- Hỗ trợ nhập Bearer JWT.
- Dễ quan sát endpoint, request, response, status code.

2.14. OWASP API Security Top 10

OWASP API Security Top 10 là bộ tham chiếu quan trọng để đánh giá an toàn cho API.

Dự án tập trung vào các nhóm lỗi để trình bày và sát với bài toán:

Nhóm lỗi	Ý nghĩa	Biện pháp xử lý trong dự án
API1: BOLA	Đổi ID tài nguyên để xem dữ liệu người khác	Ownership check cho profile, enrollment, certificate, lesson
API2: Broken Authentication	Token sai, token hết hạn, mật khẩu yếu	JWT validation, bcrypt, refresh token hash, revoke
API3: Excessive Data Exposure	Trả dư field nhạy cảm	DTO response, không trả password hash/ciphertext
API4: Unrestricted Resource Consumption	Spam request gây quá tải	Rate limiting và trả 429
API8: Security Misconfiguration	Cấu hình sai, lộ stack trace, CORS mở quá mức	CORS rõ ràng, TLS, ẩn stack trace, không hard-code secret

2.15. Docker và triển khai đóng gói

Docker giúp dự án tránh tình trạng “chạy được trên máy em nhưng không chạy được trên máy khác”. Nhờ Docker Compose, toàn bộ stack gồm:

- frontend

- backend
- database
- nginx

được khởi động theo một cấu hình nhất quán. Điều này rất hữu ích cho đồ án vì:

- Dễ chạy demo trên máy khác.
- Dễ chụp minh chứng.
- Dễ giữ nguyên môi trường khi kiểm thử.

Chương 3. Phân tích, thiết kế và triển khai hệ thống

3.1. Định hướng cấu trúc kỹ thuật của hệ thống

Trước khi đi vào chi tiết từng thành phần, nhóm xác định rõ các quyết định kỹ thuật nền tảng để hệ thống vừa đúng trọng tâm môn học, vừa đủ gọn để có thể hiện thực, kiểm thử và demo:

- Tách hệ thống thành hai phần chính: backend RESTful API tại backend/ và frontend demo tại frontend/.
- Chọn Spring Security, JWT, bcrypt, AES-GCM làm trục kỹ thuật chính cho phần xác thực, bảo vệ dữ liệu và kiểm soát truy cập.
- Giới hạn miền nghiệp vụ ở mức vừa đủ cho đề án: tài khoản người dùng, khóa học, bài học, ghi danh, chứng chỉ và vùng quản trị phê duyệt ghi danh.
- Cường chế các kiểm soát bảo mật bắt buộc ở backend; frontend chỉ đóng vai trò giao diện demo và gửi request theo đúng luồng nghiệp vụ.
- Chuẩn bị đồng thời tài liệu, script và công cụ kiểm thử để mọi kết quả bảo mật đều có thể được trình bày và kiểm chứng trực tiếp.

Bảng 3.1 tóm tắt các định hướng kỹ thuật chính khi xây dựng hệ thống.

Nội dung kỹ thuật	Định hướng lựa chọn	Cách thể hiện trong đề tài
Kiến trúc backend/frontend	Tách riêng backend API và frontend demo để làm rõ bề mặt giao tiếp	Spring Boot ở backend/ , NextJS ở frontend/ , Nginx làm reverse proxy
Nền tảng security	Dùng Spring Security và JWT để xử lý xác thực, phân quyền ở server-side	request protected đi qua filter, kiểm tra token, role và ownership tại backend

Password hashing	Không mã hóa mật khẩu, chỉ băm một chiều bằng bcrypt	áp dụng ở các luồng đăng ký, đăng nhập và đổi/đặt lại mật khẩu
Encryption dữ liệu	Mã hóa dữ liệu nhạy cảm bằng AES-GCM kèm ngữ cảnh sử dụng	áp dụng cho phone number, billing address và certificate code
Domain nghiệp vụ	Giữ phạm vi vừa đủ để phục vụ demo bảo mật API	thiết kế Course , Lesson , Enrollment , Certificate và Admin Enrollment Review thay vì mở rộng như LMS đầy đủ
Luồng mở khóa khóa học	Gắn role và trạng thái nghiệp vụ với quyền truy cập tài nguyên	STUDENT chỉ tạo PENDING , ADMIN mới chuyển sang ACTIVE , lesson chỉ mở khi enrollment đã duyệt
Demo và kiểm thử	Mọi kiểm soát cần có cách kiểm chứng bằng công cụ cụ thể	sử dụng Swagger , Postman , script demo, OWASP ZAP , integration test
Triển khai local	Ưu tiên môi trường local nhất quán, dễ chạy và dễ chụp minh chứng	Docker Compose , Nginx , https://localhost và dữ liệu mẫu phục vụ demo

3.2. Mô tả bài toán nghiệp vụ

Hệ thống mô phỏng một dịch vụ khóa học online nhỏ. Các tác nhân chính gồm:

- STUDENT: đăng ký, đăng nhập, xem khóa học, gửi yêu cầu ghi danh, xem bài học đã được mở khóa và xem certificate của chính mình.

- ADMIN: duyệt ghi danh học viên, quản lý học viên theo khóa học và xem audit log.
- Các khóa học mẫu được gắn với tài khoản quản trị nội bộ để cắt bỏ phần nghiệp vụ quản lý giảng viên không cần thiết.

Mục tiêu của hệ thống không phải làm nghiệp vụ nhiều, mà là tạo ra các điểm minh họa rõ ràng cho các cơ chế:

- Xác thực bằng JWT.
- Phân quyền role-based.
- Kiểm tra ownership.
- Mã hóa dữ liệu khi lưu trữ.
- Gắn trạng thái ghi danh với quyền truy cập bài học và certificate.

3.3. Yêu cầu hệ thống

3.3.1. Yêu cầu chức năng

Hệ thống cần đáp ứng các nhóm chức năng sau:

Nhóm Auth

- Đăng ký tài khoản sinh viên.
- Đăng nhập.
- Lấy thông tin người dùng hiện tại.
- Làm mới access token bằng refresh token.
- Đăng xuất và revoke refresh token.

Nhóm Course

- Xem danh sách khóa học công khai.
- Xem chi tiết khóa học.

Nhóm Lesson

- Xem bài học cụ thể trong một khóa học.
- Chỉ sinh viên đã enroll mới xem được nội dung đầy đủ.

Nhóm Enrollment

- Gửi yêu cầu ghi danh cho khóa học.
- Xem enrollment của chính mình.
- Xem chi tiết một enrollment.

Nhóm Certificate

- Xem danh sách certificate của chính mình.
- Xem một certificate cụ thể.

Nhóm Admin

- Xem danh sách khóa học kèm số học viên và số yêu cầu chờ duyệt.
- Xem danh sách học viên và các enrollment chờ duyệt theo từng khóa học.
- Duyệt yêu cầu ghi danh.
- Thêm hoặc xóa học viên khỏi khóa học.
- Xem hồ sơ học viên phục vụ quản trị.
- Xem audit log.

Nhóm System

- Health check.

3.3.2. Yêu cầu phi chức năng

- Giao diện đủ rõ để demo.
- Mã nguồn dễ đọc.
- Secret không hard-code trong source code.
- Hỗ trợ chạy bằng Docker Compose.
- Hỗ trợ test bằng Swagger, Postman, ZAP.

3.3.3. Yêu cầu bảo mật

- Mật khẩu phải được bcrypt hash.
- Access token phải là JWT có kiểm tra chữ ký và thời hạn.

- Dữ liệu nhạy cảm phải được AES-GCM mã hóa.
- Chỉ ADMIN mới được phê duyệt yêu cầu ghi danh.
- Lesson và certificate chỉ được truy cập khi enrollment hợp lệ.
- BOLA/IDOR phải bị chặn bằng 403.
- Có audit log cho các sự kiện an ninh.
- Có rate limiting cho endpoint nhạy cảm.
- Có HTTPS/TLS.

3.4. Thiết kế tác nhân và use case

3.4.1. Tác nhân

Tác nhân	Vai trò
Student	Học viên sử dụng hệ thống để đăng ký, gửi yêu cầu ghi danh, xem bài học và chứng chỉ
Admin	Quản trị hệ thống, duyệt ghi danh học viên, quản lý học viên theo khóa học và xem dữ liệu giám sát an ninh

3.4.2. Use case chính

Use case	Student	Admin
Đăng ký tài khoản	Có	Không
Đăng nhập	Có	Có
Xem khóa học public	Có	Có
Gửi yêu cầu ghi danh	Có	Không

Xem lesson đã mở khóa	Có	Không
Xem certificate của mình	Có	Không
Duyệt yêu cầu ghi danh	Không	Có
Thêm hoặc xóa học viên khỏi khóa học	Không	Có
Xem audit log	Không	Có

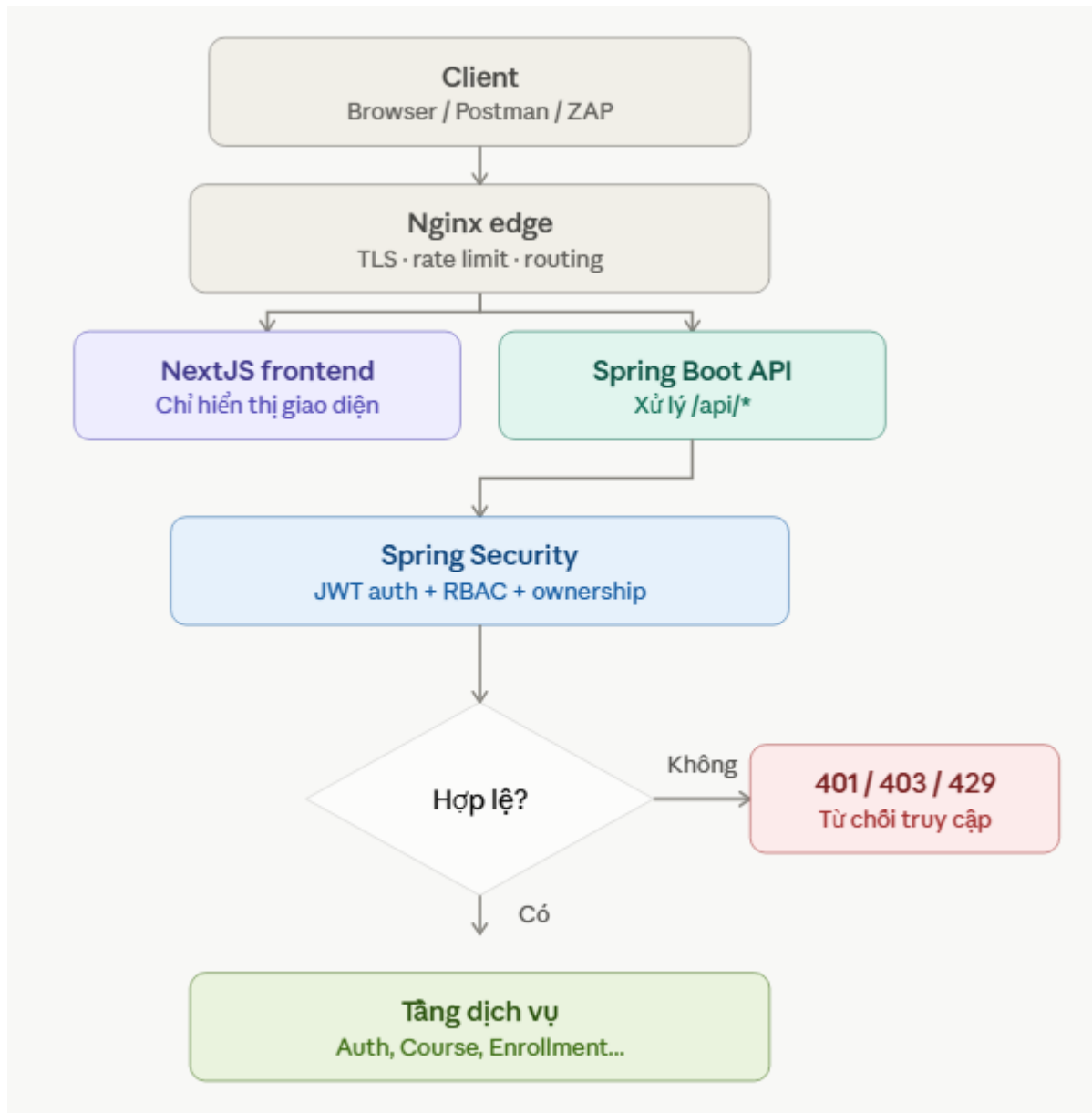
3.5. Kiến trúc giải pháp bảo mật

Phần này mô tả **kiến trúc giải pháp bảo mật** của đồ án, tức là các lớp kiểm soát được đặt ở đâu, bảo vệ ranh giới nào và tác động lên luồng dữ liệu nào. Đây không phải là phần mô tả Docker, container, port hay file cấu hình triển khai. Phần triển khai local bằng Docker Compose và Nginx được tách riêng ở Mục 3.14 để tránh nhầm lẫn giữa **security solution architecture** và **deployment scenario**.

Các sơ đồ kiến trúc bảo mật được mô tả chi tiết trong file riêng:

- docs/security-solution-architecture.md

Đồng thời, ba sơ đồ trọng tâm cũng được chèn trực tiếp trong báo cáo để phục vụ phần trình bày trước giảng viên và phần in tài liệu. Việc tách file gốc riêng giúp hình kiến trúc đi đúng hướng an toàn thông tin, thay vì biến thành sơ đồ phần mềm hoặc sơ đồ triển khai hạ tầng thuần túy.



3.5.1. Quan điểm thiết kế giải pháp

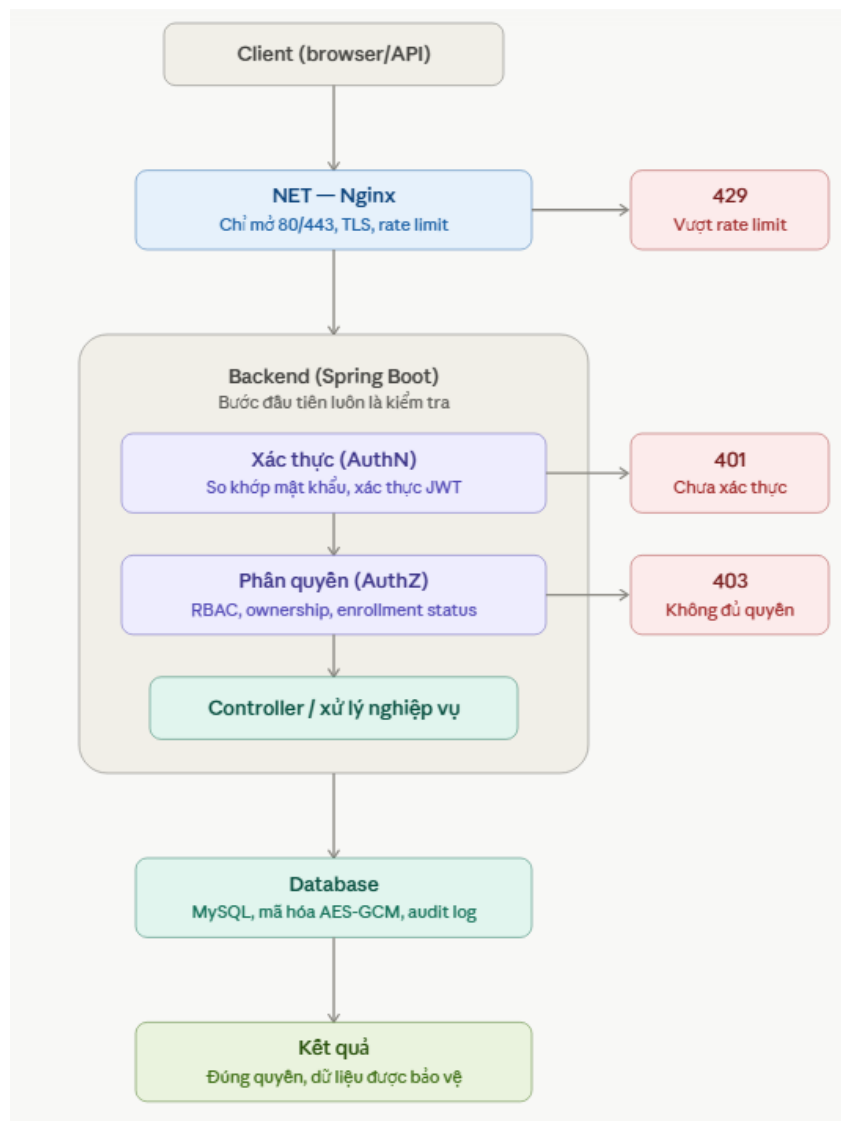
Kiến trúc giải pháp của đề tài được xây dựng theo bốn lớp phân tích chính:

- net: xem xét các biên mạng, bề mặt exposed và bảo vệ kênh truyền.
- host/service: xem xét các thành phần chạy thật, vùng tin cậy và điểm đặt policy.
- logic: xem xét các kiểm soát ở tầng ứng dụng như xác thực, phân quyền, mã hóa và giám sát.

- data flow: xem xét dữ liệu nào đi qua đâu, bị kiểm tra ở đâu và được bảo vệ bằng cơ chế gì.

Nguyên tắc xuyên suốt là:

- bắt đầu từ tài sản và rủi ro, không bắt đầu từ danh sách framework;
- cường chế kiểm soát ở server-side thay vì chỉ ở giao diện;
- tách rõ kiểm soát ở tầng biên, tầng host/service, tầng logic và tầng dữ liệu;
- phân biệt rõ “giải pháp bảo mật” với “cách triển khai local để demo”.



3.5.2. Tầng mạng (net)

Ở tầng mạng, câu hỏi trọng tâm là: hệ thống công khai những gì ra ngoài, bề mặt nào được phép đi qua Internet và lưu lượng nào phải được bảo vệ bằng TLS.

Biên mạng / điểm kiểm soát	Rủi ro chính	Kiểm soát áp dụng
Internet -> Reverse proxy	nghe lén, downgrade, gọi thẳng bề mặt nhạy cảm	HTTPS/TLS , redirect 80 -> 443 , security headers tại Nginx
Reverse proxy -> Frontend/Backend	nhầm luồng truy cập, lộ backend trực tiếp, lệch origin	reverse proxy cùng origin, route / cho frontend và /api/* cho backend
Public client -> Swagger/OpenAPI	lộ bề mặt kiểm thử nội bộ ra public	quản lý rõ host/domain public ở Nginx và Cloudflare Tunnel ; ưu tiên localhost cho demo kỹ thuật; chỉ mở public khi thực sự cần demo từ xa
Public client -> API protected	gửi trực tiếp request trái quyền hoặc spam endpoint nhạy cảm	JWT, RBAC , ownership check, rate limit cho login/register/password reset/admin API
Backend -> Database	truy cập trực tiếp database từ bên ngoài	database nằm ở mạng nội bộ của stack; không đi qua public reverse proxy

Về bản chất, tầng net của đồ án phải giải quyết ba việc: bảo vệ kênh truyền, giới hạn bề mặt exposed và đặt đúng trust boundary giữa vùng public với vùng ứng dụng nội bộ.

3.5.3. Tầng host / service (host)

Ở tầng host/service, đồ án không đi sâu theo hướng hardening hệ điều hành production-grade, nhưng vẫn cần chỉ ra rõ mỗi thành phần chạy ở đâu và có vai trò bảo mật gì.

Thành phần	Vai trò trong giải pháp bảo mật	Ghi chú
Nginx	điểm vào duy nhất từ bên ngoài, terminate TLS, thêm security headers, định tuyến đường dẫn và là nơi kiểm soát việc có mở Swagger/public tunnel hay không	đây là lớp biên, không chứa logic nghiệp vụ
Frontend NextJS	chỉ hiển thị giao diện và gửi request cùng origin	không được xem là lớp quyết định phân quyền cuối cùng
Backend Spring Boot	thực thi xác thực JWT, RBAC, ownership check, AES-GCM, audit log, rate limit và logic phê duyệt enrollment	đây là nơi đặt các kiểm soát bắt buộc
Database MySQL/H2	lưu hash, ciphertext, enrollment, certificate, refresh token và audit log	nếu dữ liệu ở đây bị lộ thì password và dữ liệu nhạy cảm vẫn không ở dạng rõ
Secret runtime	JWT_SECRET , ENCRYPTION_KEY	lấy từ biến môi trường, không hard-code trong source

Điểm cần nhấn mạnh là tầng host/service không trả lời câu hỏi “ng nghiệp vụ làm gì”, mà trả lời câu hỏi “kiểm soát nào được đặt ở thành phần nào”.

3.5.4. Tầng logic ứng dụng (logic)

Đây là tầng quan trọng nhất của đồ án vì phần lớn yêu cầu môn học nằm ở đây: xác thực, phân quyền, bảo vệ dữ liệu và xác thực thông điệp.

Nhóm kiểm soát	Thành phần logic chính	Rủi ro được xử lý
Xác thực người dùng	AuthController , AuthService , BCryptPasswordEncoder , JwtService , JwtAuthenticationFilter	đăng nhập sai, token giả mạo, token hết hạn, token sai định dạng
Phân quyền và ownership	SecurityConfig , AuthorizationService	BOLA/IDOR , leo thang đặc quyền, truy cập chéo dữ liệu
Bảo vệ dữ liệu lưu trữ	EncryptionService , AAD theo ngữ cảnh	lộ phone number, billing address, certificate code khi DB bị rò rỉ
Bảo vệ luồng ghi danh	EnrollmentService , AdminService , EnrollmentStatus	tự mở khóa bài học khi chưa được duyệt, duyệt sai role, lẫn giữa PENDING và ACTIVE
Giám sát và giảm lạm dụng	AuditLogService , RateLimitService	brute-force, spam API, thiếu khả năng truy vết

Với cách nhìn này, frontend không còn bị xem là “lớp bảo mật”. Frontend chỉ hỗ trợ trải nghiệm demo; các quyết định bảo mật thực sự đều được backend cưỡng chế.

3.5.5. Tầng luồng dữ liệu bảo mật (data flow)

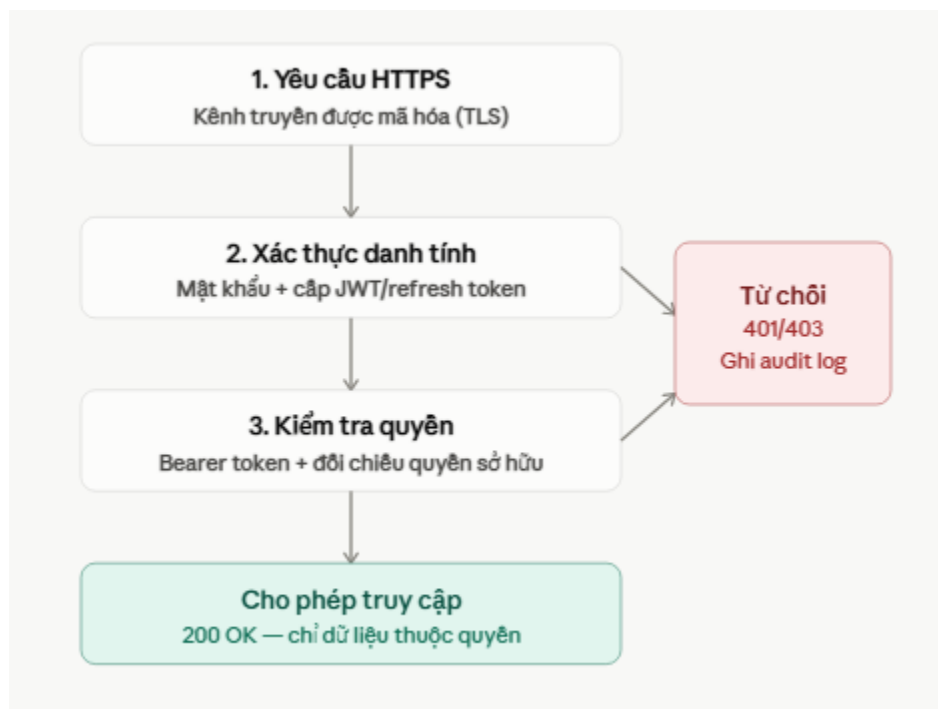
Kiến trúc bảo mật không chỉ là liệt kê thành phần, mà còn phải chỉ ra dữ liệu nào di chuyển qua những điểm nào và được kiểm soát ở đâu.

Luồng dữ liệu	Dữ liệu chính	Điểm kiểm soát bảo mật	Kết quả cần đạt
Đăng ký / đăng nhập	email, password, access token, refresh token	TLS , bcrypt , JWT signing, refresh token hash, audit log	chỉ user hợp lệ mới nhận token; password không lưu plaintext
Gọi API protected	Bearer token, object id trên URL	kiểm tra header Authorization , chữ ký JWT, exp , role, ownership	user chỉ truy cập được tài nguyên thuộc quyền
Yêu cầu ghi danh / phê duyệt	courseId , enrollmentId , status , activatedAt	RBAC , role check ở endpoint admin, kiểm tra ACTIVE trước khi mở lesson hoặc certificate	chỉ enrollment đã được duyệt mới mở khóa tài nguyên học tập
Forgot / reset password	reset token, password mới	reset token validation, revoke refresh token cũ, response không lộ thông tin email tồn tại	vòng đời tài khoản được bảo vệ đầy đủ hơn

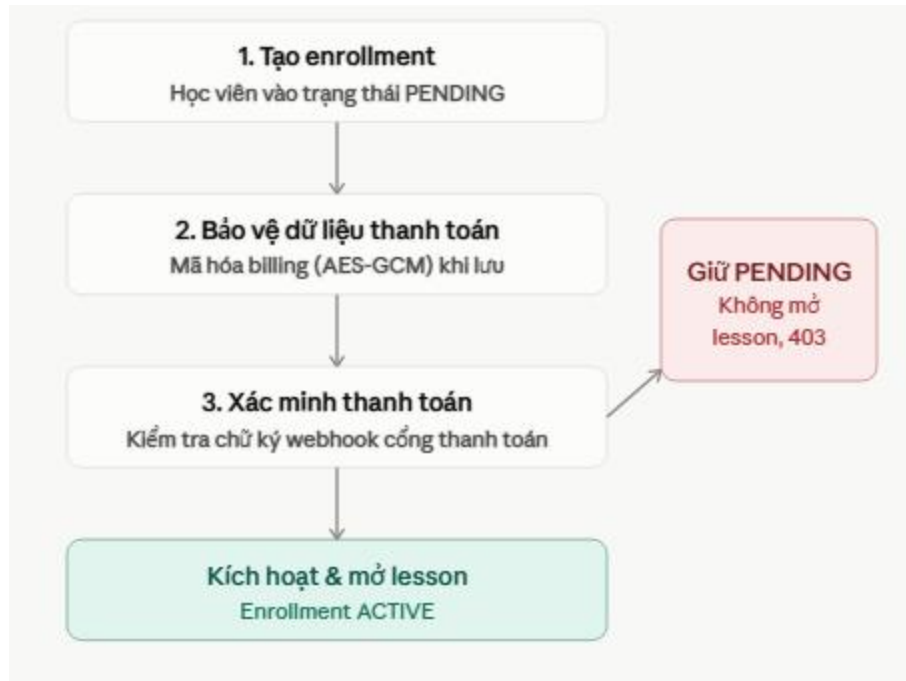
Ghi dữ liệu nhảy cảm	phone number, billing address, certificate code	AES-GCM + AAD theo ngữ cảnh nghiệp vụ	ciphertext bị sửa hoặc tráo ngữ cảnh sẽ giải mã thất bại
-------------------------	---	--	--

Từ góc nhìn data flow, có thể thấy các cơ chế mật mã của đồ án không tồn tại độc lập, mà được gắn trực tiếp vào luồng đăng nhập, luồng truy cập API, luồng ghi danh và luồng lưu trữ dữ liệu.

- Luồng 1: Đăng nhập và truy cập



- Luồng 2: Ghi danh và mở khóa lesson



3.6. Thiết kế phân quyền và scope

Phiên bản rút gọn của đồ án tập trung hai vai trò demo chính là STUDENT và ADMIN. Các khóa học mẫu được gắn với tài khoản quản trị nội bộ để tránh làm hệ thống mang dáng dấp của một nền tảng đào tạo hoàn chỉnh.

3.6.1. Role và phạm vi quyền

Role	Quyền chính
STUDENT	course:read , lesson:read , enrollment:read , certificate:read , profile:read , enrollment:request
ADMIN	course:read , course:moderate , admin:read , audit:read

3.6.2. Kiểm tra ở SecurityConfig

Tại lớp cấu hình bảo mật:

- GET /api/courses và GET /api/courses/{id} được mở public.

- POST /api/auth/register, POST /api/auth/login, POST /api/auth/refresh được phép truy cập công khai.
- POST /api/courses/{courseId}/enrollment-requests chỉ dành cho STUDENT.
- GET /api/users/{userId}/profile, GET /api/enrollments/{enrollmentId}, GET /api/certificates/{certificateId} và GET /api/courses/{courseId}/lessons/{lessonId} chỉ dành cho STUDENT.
- /api/admin/** chỉ dành cho ADMIN.

3.6.3. Kiểm tra ownership ở AuthorizationService

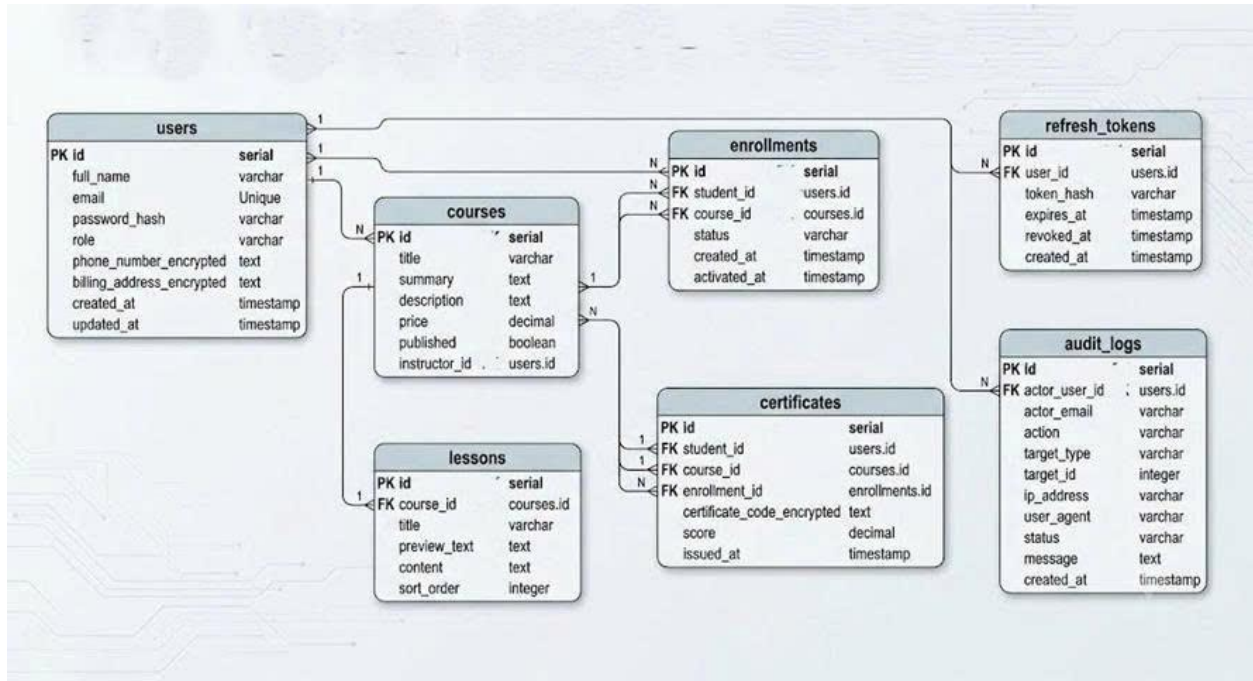
Ngoài role, hệ thống còn kiểm tra ownership cho:

- profile
- enrollment
- certificate
- lesson

Nếu không đủ quyền, service sẽ:

- Ghi ACCESS_DENIED vào audit log.
- Trả 403 Forbidden.

3.7. Thiết kế cơ sở dữ liệu



Các bảng chính của hệ thống gồm:

- users
- courses
- lessons
- enrollments
- certificates
- refresh_tokens
- audit_logs

3.7.1. Bảng users

Lưu thông tin tài khoản người dùng:

- id
- full_name
- email
- password_hash

- role
- phone_number_encrypted
- billing_address_encrypted
- created_at
- updated_at

Điểm cần chú ý:

- email là duy nhất.
- password_hash lưu bcrypt hash.
- phone_number_encrypted và billing_address_encrypted lưu ciphertext AES-GCM.

3.7.2. Bảng courses

Lưu thông tin khóa học:

- title
- summary
- description
- price
- published
- instructor_id

3.7.3. Bảng lessons

Lưu từng bài học:

- course_id
- title
- preview_text
- content
- sort_order

preview_text được dùng để hiển thị cho người chưa có quyền truy cập toàn phần.

3.7.4. Bảng enrollments

Lưu quan hệ sinh viên và khóa học:

- student_id
- course_id
- status
- created_at
- activated_at

Ràng buộc uk_enrollments_student_course đảm bảo một sinh viên không enroll trùng cùng một khóa học.

3.7.5. Bảng certificates

Lưu thông tin chứng chỉ:

- student_id
- course_id
- enrollment_id
- certificate_code_encrypted
- score
- issued_at

3.7.6. Bảng refresh_tokens

Lưu refresh token ở dạng hash:

- user_id
- token_hash
- expires_at
- revoked_at
- created_at

3.7.7. Bảng audit_logs

Lưu lịch sử sự kiện an ninh:

- actor_user_id
- actor_email
- action
- target_type
- target_id
- ip_address
- user_agent
- status
- message
- created_at

3.8. Thiết kế nhóm API chính

3.8.1. Nhóm Auth API

Method	Endpoint	Mục đích	Quyền
POST	/api/auth/register	Đăng ký tài khoản student	Public
POST	/api/auth/login	Đăng nhập	Public
POST	/api/auth/refresh	Đổi refresh token lấy access token mới	Public
POST	/api/auth/forgot-password	Yêu cầu cấp token đặt lại mật khẩu	Public

POST	/api/auth/reset-password	Đặt lại mật khẩu bằng reset token hợp lệ	Public
POST	/api/auth/logout	Revoke refresh token	Đã đăng nhập
GET	/api/auth/me	Xem hồ sơ hiện tại	Đã đăng nhập
GET	/api/users/{userId}/profile	Xem hồ sơ theo ID	Chủ sở hữu

3.8.2. Nhóm Course API

Method	Endpoint	Mục đích	Quyền
GET	/api/courses	Danh sách khóa học public	Public
GET	/api/courses/{courseId}	Chi tiết khóa học	Public

3.8.3. Nhóm Lesson API

Method	Endpoint	Mục đích	Quyền
GET	/api/courses/{courseId}/lessons/{lessonId}	Xem bài học	Student đã enroll

3.8.4. Nhóm Enrollment API

Method	Endpoint	Mục đích	Quyền
--------	----------	----------	-------

POST	/api/courses/{courseId}/enrollment-requests	Tạo hoặc làm mới enrollment PENDING	Student
GET	/api/enrollments/me	Xem enrollments của mình	Student
GET	/api/enrollments/{enrollmentId}	Xem chi tiết enrollment	Chủ sở hữu

3.8.5. Nhóm Certificate API

Method	Endpoint	Mục đích	Quyền
GET	/api/certificates/me	Xem certificate của mình	Student
GET	/api/certificates/{certificateId}	Xem một certificate	Chủ sở hữu

3.8.6. Nhóm Admin API

Method	Endpoint	Mục đích	Quyền
GET	/api/admin/courses	Xem danh sách khóa học kèm số học viên và số yêu cầu chờ duyệt	Admin

GET	/api/admin/courses/{courseId}/enrollments	Xem danh sách học viên và yêu cầu ghi danh của một khóa học	Admin
POST	/api/admin/enrollments/{enrollmentId}/approve	Duyệt yêu cầu ghi danh	Admin
POST	/api/admin/courses/{courseId}/students	Thêm trực tiếp học viên vào khóa học	Admin
DELETE	/api/admin/enrollments/{enrollmentId}	Xóa học viên khỏi khóa học hoặc từ chối yêu cầu chờ duyệt	Admin
GET	/api/admin/users/{userId}	Xem hồ sơ học viên trong khu vực quản trị	Admin
GET	/api/admin/audit-logs	Xem audit log	Admin

3.8.7. Nhóm System API

Method	Endpoint	Mục đích	Quyền
GET	/api/health	Kiểm tra trạng thái hệ thống	Public

3.9. Thiết kế các luồng nghiệp vụ và bảo mật

3.9.1. Luồng đăng ký và lưu mật khẩu

Khi người dùng đăng ký:

1. Frontend gửi fullName, email, password, phoneNumber, billingAddress.
2. Backend kiểm tra email đã tồn tại chưa.
3. Mật khẩu được băm bằng BCryptPasswordEncoder.
4. phoneNumber và billingAddress được mã hóa bằng EncryptionService.
5. User được lưu với role mặc định STUDENT.
6. Audit log ghi REGISTER_SUCCESS.
7. Hệ thống phát hành access token và refresh token.

Luồng này minh họa rõ việc kết hợp cả hashing và encryption trong cùng một request.

3.9.2. Luồng đăng nhập và phát hành JWT

Khi người dùng đăng nhập:

1. Backend tìm user theo email.
2. Gọi passwordEncoder.matches() để kiểm tra mật khẩu.
3. Nếu sai, ghi LOGIN_FAILED vào audit log và trả 401.
4. Nếu đúng, hệ thống sinh access token JWT.
5. JWT chứa:
 - sub
 - email
 - role
 - scope
 - token_type

- iat
- exp

6. Hệ thống sinh refresh token ngẫu nhiên.

7. Refresh token được băm SHA-256 trước khi lưu vào bảng refresh_tokens.

3.9.3. Luồng xác thực request protected

JwtAuthenticationFilter đọc header Authorization và thực hiện:

1. Kiểm tra prefix Bearer.
2. Parse JWT.
3. Kiểm tra chữ ký bằng secret key.
4. Kiểm tra token_type = access.
5. Kiểm tra sub, email, role.
6. Nếu hợp lệ, tạo Authentication cho Spring Security.
7. Nếu không hợp lệ, từ chối request và ghi sự kiện phù hợp.

3.9.4. Luồng refresh token

Khi client gọi refresh:

1. Backend hash refresh token nhận được.
2. Tìm token hash trong database.
3. Kiểm tra token chưa bị revoke.
4. Kiểm tra token chưa hết hạn.
5. Revoke token cũ.
6. Cấp cập token mới.
7. Ghi TOKEN_REFRESHED vào audit log.

Luồng này giúp hạn chế việc một refresh token bị tái sử dụng nhiều lần.

3.9.5. Luồng quên mật khẩu và đặt lại mật khẩu

Khi người dùng quên mật khẩu:

1. Người dùng gửi email tới POST /api/auth/forgot-password.
2. Backend áp dụng rate limit cho endpoint này để tránh spam.
3. Hệ thống tìm tài khoản theo email.
4. Nếu tài khoản tồn tại, backend sinh một reset token ngẫu nhiên.
5. Reset token được băm SHA-256 trước khi lưu vào bảng password_reset_tokens.
6. Ở chế độ demo local, hệ thống trả thêm demoResetToken để thuận tiện kiểm thử; ở môi trường production, token này cần được gửi qua email thay vì trả thẳng ra API.
7. Audit log ghi sự kiện PASSWORD_RESET_REQUESTED.

3.9.6. Luồng yêu cầu ghi danh và phê duyệt ghi danh

Khi sinh viên gửi yêu cầu ghi danh:

1. Frontend gọi POST /api/courses/{courseId}/enrollment-requests.
2. Backend xác thực JWT và kiểm tra role hiện tại phải là STUDENT.
3. Hệ thống tìm khóa học theo courseId và kiểm tra khóa học đang được publish.
4. Nếu enrollment đã ở trạng thái ACTIVE, backend trả lỗi xung đột để tránh ghi danh trùng.
5. Nếu enrollment đã ở trạng thái PENDING, backend trả lại thông tin hiện tại thay vì tạo bản ghi mới.
6. Nếu chưa tồn tại, backend tạo enrollment mới với trạng thái PENDING.
7. Kết quả trả về gồm enrollmentId, courseId, courseTitle, status và thông báo để frontend hiển thị đúng trạng thái chờ duyệt.

Khi quản trị viên phê duyệt yêu cầu:

1. Admin mở danh sách enrollment theo khóa học tại GET `/api/admin/courses/{courseId}/enrollments`.
2. Từ giao diện quản trị hoặc Swagger/Postman, admin gọi POST `/api/admin/enrollments/{enrollmentId}/approve`.
3. Backend xác thực JWT, role ADMIN và rate limit cho khu vực quản trị.
4. Hệ thống kiểm tra enrollment hiện tại phải ở trạng thái PENDING.
5. Enrollment được chuyển sang ACTIVE và cập nhật `activatedAt`.
6. Audit log ghi `ENROLLMENT_APPROVED`.
7. Từ thời điểm này, sinh viên mới có thể xem nội dung lesson đầy đủ và nhận certificate khi dữ liệu liên quan đã tồn tại.

Nếu yêu cầu chưa được duyệt, mọi truy cập lesson protected hoặc certificate tương ứng vẫn bị backend từ chối theo đúng role, ownership và trạng thái nghiệp vụ.

3.9.7. Luồng chống BOLA/IDOR

Ví dụ với GET `/api/certificates/{certificateId}`:

1. Người dùng đã đăng nhập gửi request với `certificateId`.
2. Backend lấy certificate theo ID.
3. `AuthorizationService.assertCanViewCertificate()` kiểm tra `certificate.student.id == currentUser.id`.
4. Nếu không thỏa, ghi `ACCESS_DENIED`.
5. Trả 403 Forbidden.

Luồng tương tự được áp dụng cho:

- profile
- enrollment
- lesson

3.9.8. Luồng xem bài học

Đối với endpoint lesson:

1. Backend xác định bài học thuộc khóa học nào.
2. Kiểm tra người dùng hiện tại đã có enrollment ACTIVE hay chưa.
3. Nếu chưa đủ quyền, chỉ cho phép xem preview ở chi tiết khóa học, còn truy cập trực tiếp lesson sẽ bị từ chối 403.
4. Nếu đủ quyền, trả đầy đủ content.

Đây là điểm minh họa tốt cho việc liên kết bảo mật với nghiệp vụ thật.

3.10. Triển khai các kỹ thuật mật mã trong mã nguồn

3.10.1. Triển khai JWT trong JwtService

JwtService thực hiện:

- Dẫn xuất khóa ký từ JWT_SECRET bằng SHA-256.
- Phát hành access token bằng thư viện jjwt.
- Parse và validate token.

Từ chối:

- token hết hạn,
- token sai chữ ký,
- token malformed,
- token thiếu subject, email, role,
- token có token_type khác access.

3.10.2. Triển khai bcrypt trong SecurityConfig và AuthService

SecurityConfig khai báo PasswordEncoder là BCryptPasswordEncoder.

AuthService dùng encoder này trong:

- register(): encode password trước khi lưu.

- login(): matches giữa password người dùng nhập và hash đã lưu.

3.10.3. Triển khai AES-GCM trong EncryptionService

EncryptionService được tách thành service độc lập để tái sử dụng cho nhiều loại dữ liệu.

Service này cung cấp các hàm riêng cho:

- dữ liệu user,
- dữ liệu certificate,
- dữ liệu demo.

3.10.4. Triển khai kiểm soát ghi danh trong EnrollmentService và AdminService

EnrollmentService:

- chỉ cho STUDENT tạo yêu cầu ghi danh,
- kiểm tra khóa học tồn tại và đang publish,
- tránh tạo trùng nếu enrollment đã ACTIVE,
- trả lại trạng thái cũ nếu yêu cầu đã PENDING.

AdminService:

- chỉ cho ADMIN duyệt enrollment,
- chỉ chấp nhận duyệt khi trạng thái hiện tại là PENDING,
- chuyển enrollment sang ACTIVE,
- ghi audit log ENROLLMENT_APPROVED.

3.11. Triển khai rate limiting và audit log

3.11.1. Rate limiting

RateLimitService trong dự án sử dụng chiến lược cửa sổ cố định một phút (fixed window), lưu bộ đếm trong ConcurrentHashMap. Dù đây chưa phải kiến trúc phân tán production-grade, nhưng rất phù hợp với đồ án vì:

- Dễ đọc.

- Dễ kiểm chứng.
- Dễ demo 429 Too Many Requests.

Các giá trị mặc định được lấy từ cấu hình:

- `RATE_LIMIT_LOGIN_PER_MINUTE = 5`
- `RATE_LIMIT_REGISTER_PER_MINUTE = 3`
- `RATE_LIMIT_PASSWORD_RESET_PER_MINUTE = 3`
- `RATE_LIMIT_ADMIN_PER_MINUTE = 30`

3.11.2. Audit log

AuditLogService được gọi xuyên suốt từ nhiều nơi:

- AuthService
- AuthorizationService
- SecurityConfig
- EnrollmentService
- AdminService
- RateLimitService

Nhờ đó, hệ thống có thể ghi lại cả:

- sự kiện thành công,
- sự kiện thất bại,
- hành vi bất thường,
- hành vi bị chặn.

3.12. Cấu hình bảo mật ở lớp ứng dụng và reverse proxy

3.12.1. CORS

CorsConfig đọc các origin được phép từ biến môi trường `CORS_ALLOWED_ORIGINS`.

Mặc định:

```
https://localhost,http://localhost:3000
```

3.12.2. Error handling

Trong application.properties, hệ thống cấu hình:

- server.error.include-stacktrace=never
- server.error.include-message=never

Mục tiêu là giảm rò rỉ thông tin nội bộ ở môi trường runtime.

3.12.3. Security headers

Ở lớp reverse proxy, Nginx thêm các security header sau:

- Strict-Transport-Security
- Content-Security-Policy
- X-Content-Type-Options
- X-Frame-Options
- Referrer-Policy

Các header này góp phần xử lý nhóm lỗi Security Misconfiguration.

3.13. Thiết kế giao diện frontend phục vụ demo

Frontend được xây dựng tối giản để tập trung vào việc trình diễn luồng bảo mật. Các trang chính gồm:

- Home page
- Login page
- Register page
- Forgot password page
- Reset password page
- Courses page
- Course detail page
- Lesson detail page
- Profile / Certificate page

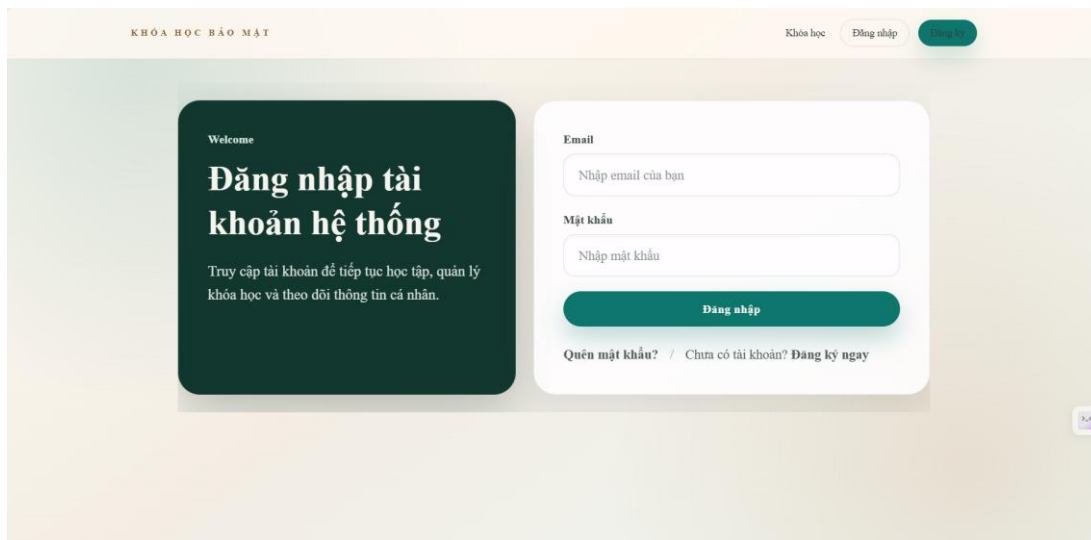
- Admin Enrollment Review page
- Admin Audit Logs page

Một số điểm thiết kế:

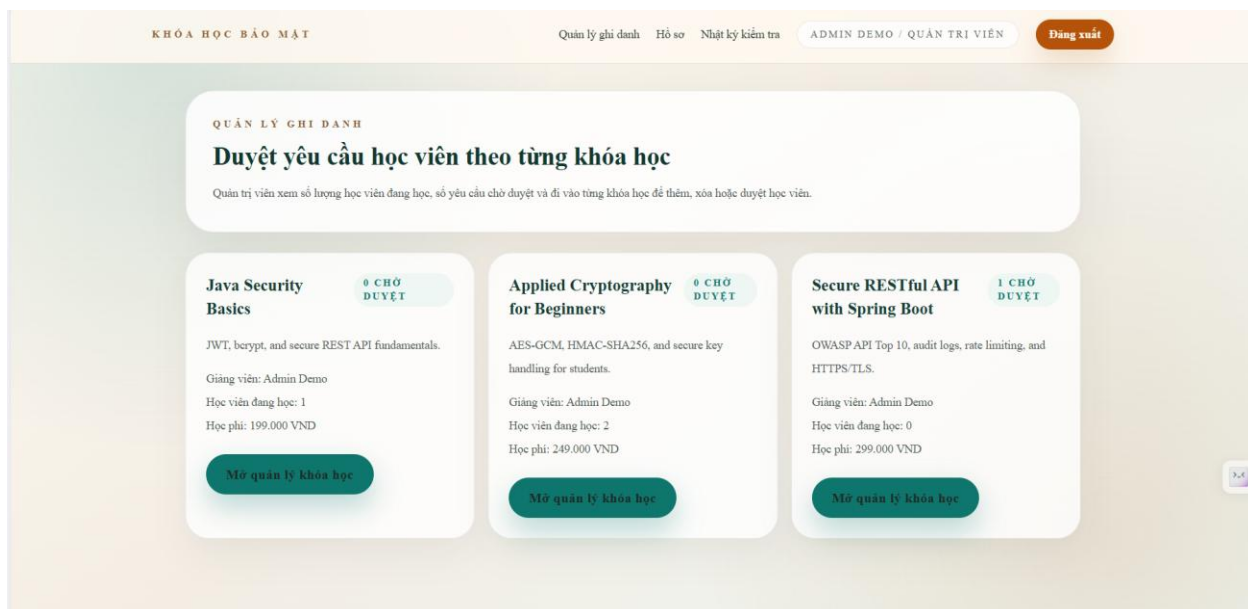
- Token được lưu ở mức demo để dễ kiểm thử.
- Các request đi qua cùng origin `https://localhost`.
- Giao diện ưu tiên thông báo dễ hiểu cho người dùng thay vì phơi bày trực tiếp các mã lỗi kỹ thuật như 401, 403, 429.
- Frontend không hiển thị các nội dung kiểm thử nội bộ như Swagger, checklist OWASP ZAP, gợi ý BOLA/IDOR hay các nhãn kỹ thuật chỉ phục vụ demo bảo mật.



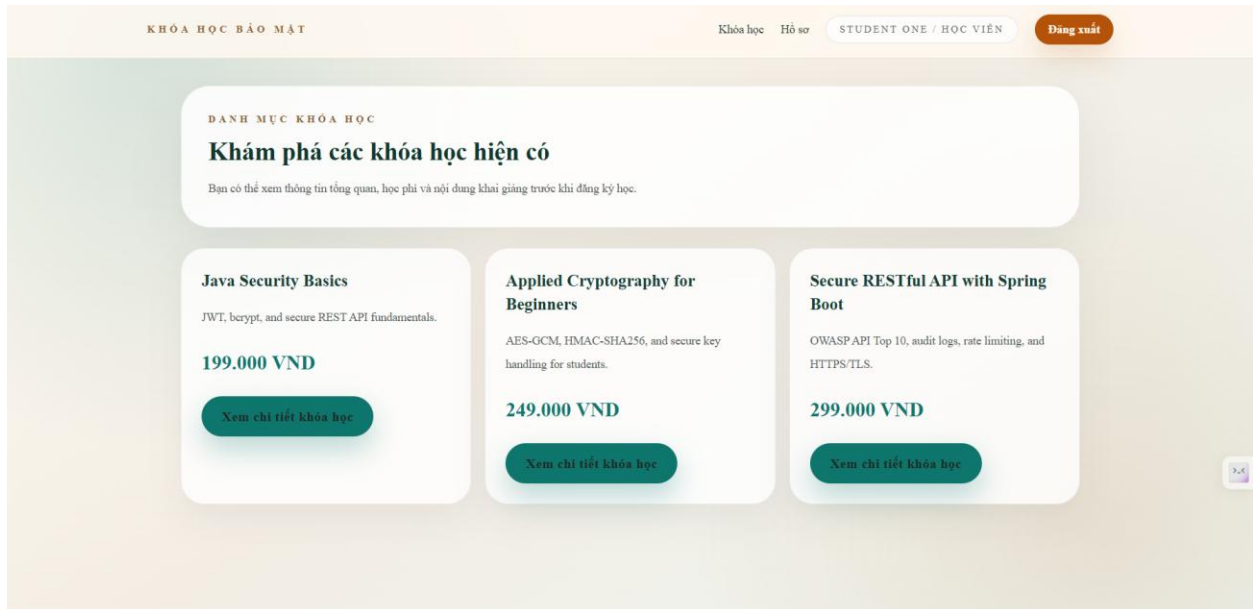
Giao diện trang chủ



Giao diện đăng nhập



Menu chính khi đăng nhập bằng tài khoản admin



Menu chính khi đăng nhập bằng tài khoản student

3.14. Docker hóa và kịch bản triển khai local

Mục này mô tả cách hiện thực hóa hệ thống trên môi trường demo local bằng Docker Compose, Nginx, HTTPS và các port cụ thể.

Stack được tổ chức qua docker-compose.yml với các thành phần:

- backend
- frontend
- database
- nginx

3.14.1. Cổng và truy cập

Theo cấu hình hiện tại:

- HTTP local: 80
- HTTPS local: 443
- Frontend truy cập qua: <https://localhost>
- Swagger truy cập qua: <https://localhost/swagger-ui.html>
- MySQL host port: 3307

3.14.2. Biến môi trường chính

Các biến môi trường quan trọng:

- JWT_SECRET
- JWT_ACCESS_TOKEN_EXPIRE_MINUTES
- JWT_REFRESH_TOKEN_EXPIRE_DAYS
- ENCRYPTION_KEY
- CORS_ALLOWED_ORIGINS
- APP_API_BASE_URL

3.15. Dữ liệu mẫu phục vụ demo

Hệ thống có dữ liệu seed để tiết kiệm thời gian trình bày:

Tài khoản	Vai trò	Mật khẩu
student1@example.com	STUDENT	Password123!
student2@example.com	STUDENT	Password123!
admin@example.com	ADMIN	Admin123!

Nguồn seed hiện chỉ giữ các tài khoản STUDENT và ADMIN. Các khóa học mẫu được gắn trực tiếp với tài khoản quản trị nội bộ để mô hình dữ liệu gọn hơn và đúng trọng tâm bảo mật.

Dữ liệu nghiệp vụ seed:

- student1 đã enroll khóa Java Security Basics.
- student2 đã enroll khóa Applied Cryptography for Beginners.
- student1 có một enrollment PENDING cho khóa Secure RESTful API with Spring Boot để demo luồng admin phê duyệt ghi danh.

3.16. Bảng tổng hợp một số kịch bản demo tấn công/phòng thủ

Kịch bản	Mục tiêu	Kết quả mong đợi
Xem password trong database	Chứng minh không lưu plaintext	Thấy bcrypt hash
Sửa payload JWT	Kiểm tra tính toàn vẹn token	Backend trả 401
Dùng JWT hết hạn	Kiểm tra expiry	Backend trả 401
Yêu cầu quên mật khẩu và đặt lại bằng token hợp lệ	Kiểm tra vòng đời tài khoản	Mật khẩu mới có hiệu lực, refresh token cũ bị revoke
Đổi certificateId sang dữ liệu người khác	Demo BOLA/IDOR	Backend trả 403 và ghi audit log
Gửi yêu cầu ghi danh rồi thử xem lesson ngay	Demo tách PENDING với ACTIVE	Lesson vẫn bị khóa, backend trả 403
Admin duyệt enrollment PENDING	Demo role-based workflow	Enrollment chuyển ACTIVE, lesson được mở
Spam login sai nhiều lần	Demo rate limiting	Backend trả 429
Dùng student gọi /api/admin/audit-logs	Demo RBAC	Backend trả 403

3.17. Kết quả đạt được

Qua quá trình phân tích, thiết kế và triển khai, hệ thống đã đạt được các kết quả chính:

- Xây dựng domain khóa học online tối giản, phù hợp đề tài.
- Tích hợp JWT access token và refresh token hash.
- Áp dụng bcrypt đúng mục đích cho mật khẩu.
- Áp dụng AES-GCM cho nhiều loại dữ liệu nhạy cảm.
- Xây dựng luồng STUDENT gửi yêu cầu ghi danh và ADMIN phê duyệt rõ ràng.
- Kiểm soát truy cập theo role và ownership.
- Ghi audit log cho các hành vi quan trọng.
- Áp dụng rate limiting cho các endpoint nhạy cảm.
- Thiết lập Docker + Nginx TLS để chạy local qua HTTPS.
- Chuẩn bị tài liệu Postman, ZAP và các kịch bản demo phục vụ kiểm thử.

3.18. Hạn chế của hệ thống

Hệ thống vẫn còn một số hạn chế:

- Rate limit hiện dùng bộ nhớ trong tiến trình, chưa phù hợp hệ phân tán nhiều node.
- TLS local đang dùng self-signed certificate.
- Frontend lưu token theo cách thuận tiện cho demo, chưa phải mô hình tối ưu production.
- Chưa có cơ chế xác thực đa yếu tố cho tài khoản quản trị.
- Chưa có key rotation cho JWT và AES.
- Chưa có hệ thống giám sát thời gian thực hoặc SIEM.

3.19. Hướng phát triển

Nếu tiếp tục mở rộng, hệ thống có thể phát triển theo các hướng:

- Chuyển rate limiting sang Redis.
- Dùng HTTP-only cookies hoặc cơ chế lưu token an toàn hơn.
- Triển khai key rotation cho secret.
- Bổ sung xác thực đa yếu tố cho tài khoản quản trị hoặc cho các thao tác nhạy cảm.

- Xây dựng cơ chế tổng hợp và trực quan hóa audit log ở tầng giám sát riêng nếu tiếp tục mở rộng sau này.
- Tích hợp pipeline CI/CD có SAST và DAST.

Chương 4. Thực nghiệm, demo và đánh giá hệ thống

4.1. Môi trường và điều kiện thực nghiệm

Toàn bộ phần thực nghiệm trong đề án được thực hiện trên stack Docker của dự án. Các thành phần chính gồm:

Thành phần	Mô tả
frontend	Ứng dụng NextJS phục vụ giao diện demo
backend	Ứng dụng Spring Boot cung cấp RESTful API
database	MySQL lưu trữ dữ liệu người dùng, khóa học, enrollment, certificate, refresh token và audit log
nginx	Reverse proxy chịu trách nhiệm terminate TLS và chuyển hướng HTTP sang HTTPS

Các cổng và địa chỉ truy cập chính trong quá trình thực nghiệm:

Mục	Địa chỉ
Frontend local	https://hackerlo.online
Health check	https:// hackerlo.online/api/health
Swagger UI	https:// hackerlo.online/swagger-ui.html
OpenAPI JSON	https:// hackerlo.online/v3/api-docs

4.2. Dữ liệu mẫu và công cụ phục vụ thực nghiệm

4.2.1. Tài khoản mẫu

Email	Vai trò	Mật khẩu	Mục đích sử dụng
student1@example.com	STUDENT	Password123!	Demo đăng nhập, chứng chỉ, bài học và yêu cầu ghi danh
student2@example.com	STUDENT	Password123!	Demo BOLA/IDOR và so sánh quyền
admin@example.com	ADMIN	Admin123!	Demo audit log và quản trị enrollment

4.2.2. Công cụ sử dụng

Công cụ	Vai trò trong thực nghiệm
Swagger UI	Gọi trực tiếp REST API, hiển thị request/response và status code
Postman	Thực hiện các kịch bản giữ nhiều token cùng lúc, đặc biệt là BOLA, enrollment request và admin approval
PowerShell	Gửi request hàng loạt, giải mã JWT, truy vấn database, chạy script demo
MySQL trong Docker	Kiểm tra dữ liệu thực lưu trong cơ sở dữ liệu
OWASP ZAP	Quét baseline và kiểm tra bề mặt web/API qua HTTPS

4.2.3. Điều kiện chuẩn bị trước khi demo

Trước khi bắt đầu thực nghiệm, cần khởi động stack và bật tunnel:

```
docker compose up --build -d  
.\scripts\start-public-tunnel.ps1
```

Sau đó kiểm tra nhanh:

```
curl.exe -k https://hackerlo.online/api/health
```

Khi kết quả trả về 200 OK hoặc trạng thái UP, có thể tiếp tục các bài thực nghiệm tiếp theo.

4.3. Kịch bản 1: Kiểm chứng băm mật khẩu bằng bcrypt

4.3.1. Mục tiêu

Kịch bản này nhằm chứng minh ba điểm:

- Mật khẩu không được lưu dưới dạng rõ;
- Backend vẫn xác thực đúng bằng mật khẩu gốc;
- Người quan sát database không thể đọc ngược ra mật khẩu.

4.3.2. Cách thực hiện

Công cụ nên dùng: Swagger UI để login, PowerShell để query database.

Tài khoản nên dùng: student1@example.com / Password123!.

Các bước thực hiện như sau:

1. Đăng nhập.

Request body required

application/json

```
{
  "email": "student@example.com",
  "password": "Password123!"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'https://hackerlo.online/api/auth/login' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "student@example.com",
    "password": "Password123!"
  }'
```

Request URL

https://hackerlo.online/api/auth/login

2. Ghi nhận response đăng nhập thành công.

```
Request URL
https://hackerlo.online/api/auth/login

Server response

Code Details

200

Response body
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cGU6IjwiZXN0ZW8yZm9udDpyZXIjXzAwbWVldG9rZW5fdmlwZS16ImFmfY2VzcyIjOnJhdC16MTk0M3Q0NDNsOCCaZXBwIjoxNDg4MjE4fQ.",
  "refreshToken": "qzoinsxTj8cu0CIYY4Cu0MaSygnjN8EdexQDEHlJEXDbSlugBGZ4JCkdVdflpf",
  "tokenType": "bearer",
  "accessTokenExpiresInSeconds": 900,
  "refreshTokenExpiresInSeconds": 604800,
  "role": "STUDENT",
  "scope": "course:read lesson:read enrollment:read certificate:read profile:read enrollment:request",
  "user": {
    "id": 1,
    "fullName": "Student One",
    "email": "student1@example.com",
    "role": "STUDENT",
    "phoneNumber": "0901000001",
    "billingAddress": "123 Student Street",
    "createdAt": "2026-06-25T11:14:19Z"
  }
}
```

3. Mở PowerShell và truy vấn bảng users.

```
mysql> SELECT id,email,password_hash,role FROM users;
```

id	email	password_hash	role
1	student1@example.com	\$2a\$10\$Gri./4ebd102/xBvRLxd6us7i3VJ47e/Zt9SBjDliqcaWaEXey8q.	STUDENT
2	student2@example.com	\$2a\$10\$IGOYnYcNrP9at4ecsv63ke6KXFP700EdRkDeoxqULpXBSnUIHHB/q	STUDENT
3	admin@example.com	\$2a\$10\$RW1K7IpNiklLMu3vNX0UmeJjG759B/DLhQrap/RTcwtEmDKGa027K	ADMIN
4	postman.student@example.com	\$2a\$10\$0SoYwM40yoxD9sz7Y04ePOZUfkdzAm/f83Qzu99L0fyvLfC0CAVvy	STUDENT
5	postman1.student@example.com	\$2a\$10\$F8udbdzm3o4F61LA9xo0IesnNT0iKc0C6WcfCucfmmN.sYlwhsXmVS	STUDENT

```
5 rows in set (0.00 sec)
```

4. Thử đăng nhập bằng mật khẩu sai.

```
{
  "email": "student1@example.com",
  "password": "Password123!@@"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'https://hackerlo.online/api/auth/login' \
  -H 'accept: */*' \
  -H 'content-type: application/json' \
  -d '{
    "email": "student1@example.com",
    "password": "Password123!@@"
  }'
```

Request URL

```
https://hackerlo.online/api/auth/login
```

Server response

Code	Details
401	Error: response status is 401

Response body

```
{
  "timestamp": "2026-06-26T15:31:41.533044158Z",
  "status": 401,
  "error": "Unauthorized",
  "message": "Invalid email or password.",
  "path": "/api/auth/login"
}
```

Download

5. Kiểm tra auditlog

```
mysql> SELECT actor_email,action,status,created_at FROM audit_logs ORDER BY created_at DESC LIMIT 10;
```

actor_email	action	status	created_at
student1@example.com	LOGIN_FAILED	FAILED	2026-06-26 15:31:42
admin@example.com	LOGIN_FAILED	FAILED	2026-06-26 15:30:58
admin@example.com	LOGIN_SUCCESS	SUCCESS	2026-06-26 15:26:34
student1@example.com	LOGIN_SUCCESS	SUCCESS	2026-06-26 15:22:50
student1@example.com	LOGIN_SUCCESS	SUCCESS	2026-06-26 15:21:59
student2@example.com	LOGIN_FAILED	FAILED	2026-06-26 15:15:13
student2@example.com	LOGIN_FAILED	FAILED	2026-06-26 15:14:59
student1@example.com	LOGIN_FAILED	FAILED	2026-06-26 15:14:55
admin@example.com	COURSE_ENROLLMENT_DETAIL_VIEWED	SUCCESS	2026-06-26 15:14:09
admin@example.com	COURSE_ENROLLMENT_OVERVIEW_VIEWED	SUCCESS	2026-06-26 15:13:56

10 rows in set (0.00 sec)

4.4.3. Kết quả quan sát được

Kết quả thực tế cho thấy:

- Trường password_hash là chuỗi bcrypt bắt đầu bằng \$2...;
- Cột này không chứa Password123!;
- Đăng nhập đúng trả 200 OK;
- Đăng nhập sai trả 401 Unauthorized;
- audit log ghi nhận LOGIN_SUCCESS và LOGIN_FAILED.

4.4.4. Nhận xét

Kịch bản này chứng minh hệ thống đã dùng đúng cơ chế mật mã cho mật khẩu. Đây là điểm rất quan trọng, vì trong thực hành bảo mật:

- mật khẩu phải được băm chứ không được mã hóa;
- bcrypt phù hợp hơn các hash nhanh như MD5 hoặc SHA-1 khi lưu password;
- việc dùng matches() cho phép xác thực mà không cần giải mã bất cứ thứ gì.

4.4. Kịch bản 2: Kiểm chứng JWT, Bearer Token, refresh token và logout

4.4.1. Mục tiêu

Kịch bản này dùng để xác nhận:

- access token thật sự là JWT và chứa các claim cần thiết;
- backend chỉ chấp nhận đúng Bearer token;
- token bị sửa payload sẽ bị từ chối;
- token hết hạn sẽ bị từ chối;
- refresh token có thể dùng để quay vòng phiên;
- refresh token bị revoke sau logout thì không dùng lại được.

4.4.2. Các bước thực nghiệm chính

Kịch bản được chia thành các bài nhỏ:

1. Đăng nhập qua POST /api/auth/login.

The screenshot displays a REST client interface with a POST request to `/api/auth/login`. The request body is a JSON object containing email and password. The response is a 200 status code with a detailed JSON body containing access and refresh tokens, expiration times, role, scope, and user details.

POST /api/auth/login Authenticate with email and password to receive JWT and refresh token

Parameters: No parameters

Request body: `application/json`

```
{
  "email": "student1@example.com",
  "password": "Password123!"
}
```

Execute Clear

Server response

Code: 200 Details

Response body

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiJ9.eyJzdWUiOiIxIiwiaWwiOiJhZW1haWwiOiJzdHVkZW50MUBleGFtcGxlLnVbSIsInVjbGU0IjTVFVERU5UIiwic2NvcGU0Ijpb3Vyc2U6cmVhZCBsZXNzb246cmVhZCB1bnVbGxtZW50OnJlYWQgY2VydGlmaWNhdGU6cmVhZCBwcm9maWxlOnJlYWQgZW5yb2xsbWVudDpyZXF1ZXN0IiwidG9rZW5fdHlwZSI6ImFjY2VzcyIsIm1hdCI6MTc4MjQ5MDgyMCwiZXhwIjojNzgyNDkxNzIwIiwiaWF0IjoiF8JUoy2kzMm41SUh0IjNF3tZNVUIl7XTS3pz2qJB8Y",
  "refreshToken": "YeaO2XjuNail3VB7wtu9r6b8VHPwby-7b80ZvLKHA-kyVkvckDXM4ttCYiCANA",
  "tokenType": "Bearer",
  "accessTokenExpiresInSeconds": 900,
  "refreshTokenExpiresInSeconds": 604800,
  "role": "STUDENT",
  "scope": "course:read lesson:read enrollment:read certificate:read profile:read enrollment:request",
  "user": {
    "id": 1,
    "fullName": "Student One",
    "email": "student1@example.com",
    "role": "STUDENT",
    "phoneNumber": "0901000001",
    "billingAddress": "123 Student Street",
    "createdAt": "2026-06-25T11:14:19Z"
  }
}
```

Download

"accessToken":

"eyJhbGciOiJIUzI1NiJ9.eyJzdWUiOiIxIiwiaWwiOiJhZW1haWwiOiJzdHVkZW50MUBleGFtcGxlLnVbSIsInVjbGU0IjTVFVERU5UIiwic2NvcGU0Ijpb3Vyc2U6cmVhZCBsZXNzb246cmVhZCB1bnVbGxtZW50OnJlYWQgY2VydGlmaWNhdGU6cmVhZCBwcm9maWxlOnJlYWQgZW5yb2xsbWVudDpyZXF1ZXN0IiwidG9rZW5fdHlwZSI6ImFjY2VzcyIsIm1hdCI6MTc4MjQ5MDgyMCwiZXhwIjojNzgyNDkxNzIwIiwiaWF0IjoiF8JUoy2kzMm41SUh0IjNF3tZNVUIl7XTS3pz2qJB8Y"

2. Dùng accessToken gọi GET /api/auth/me.

The screenshot shows the 'Online Course Security API' interface. A modal window titled 'Available authorizations' is open, displaying 'bearerAuth (http, Bearer)' with a value 'eyJhbGciOiJIUzI1NiJ9.eyJzdWl'. Below the modal, a list of API endpoints is visible, including '/api/admin/enrollments/{enrollmentId}', '/api/admin/courses/{courseId}', and '/api/admin/users/{userId}'. Below the list, a 'Server response' section shows a 200 status code and a JSON response body:

```
{
  "id": 1,
  "fullName": "Student One",
  "email": "student1@example.com",
  "role": "STUDENT",
  "phoneNumber": "0901000001",
  "billingAddress": "123 Student Street",
  "createdAt": "2026-06-25T11:14:19Z"
}
```

3. Sửa payload token để kiểm tra chữ ký sai.

The screenshot shows a JWT token editor interface. The 'Algorithm & Token Type' section shows 'alg': 'HS256'. The 'Encoded JWT' section shows a long base64-encoded string. The 'Payload' section shows a JSON object with fields: 'sub': '1', 'email': 'student1@example.com', 'role': 'ADMIN', 'scope': 'course:read lesson:read enrollment:read certificate:read profile:read enrollment:request', 'token_type': 'access', 'iat': 1782490820, and 'exp': 1782491720. The 'role' field is highlighted with a red box.

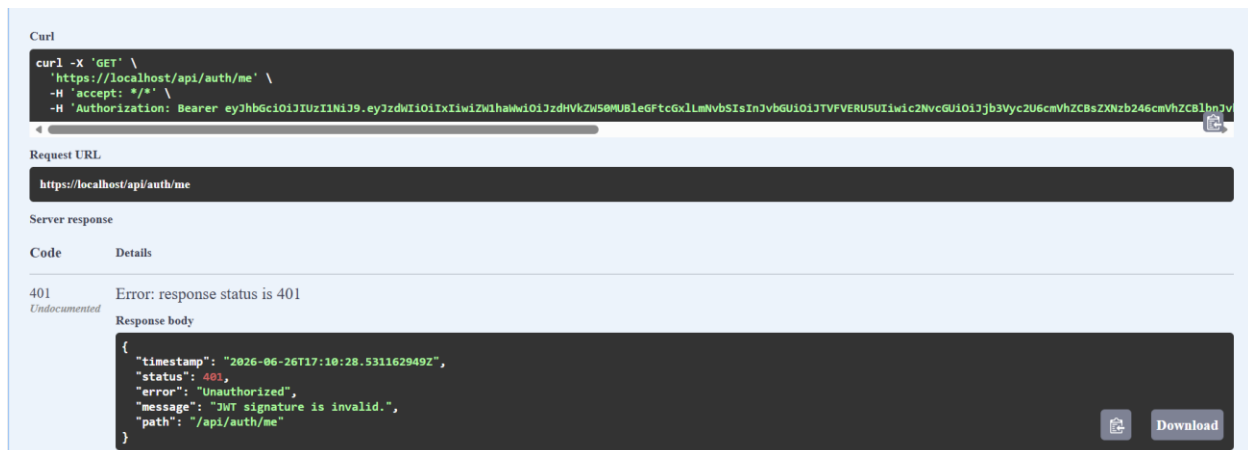
“eyJhbGciOiJIUzI1NiJ9.eyJzdWUiOiIxIiwiaWwiZW1haWwiOiJzdHVkZW50MUBleGFtcGxlLmNvbSIsInJvbGUiOiJBRE1JTilzInNjb3BlIjoieY291cnNlOnJlYWQgbGVzc29uOnJlYWQgZW5yb2xsbWVudDpyZWFKIGNlcnRpZmljYXRlOnJlYWQgcHJvZmlsZTpyZWFKIGVucm9sbG1lbnQ6cmVxdWVzdCIsInRva2VuX3R5cGUiOiJhY2Nlc3MiLCJpYXQiOiJ0eXQpOjE3ODI0OTA4MjAsImV4cCI6MTc0MjQ5MTcyMH0.ijgDzMq6LMgfvlmA88D1qyyUx41TT4pA-FYqIFpgrDc”

Server response	
Code	Details
401 <i>Undocumented</i>	Error: response status is 401 Response body <pre>{ "timestamp": "2026-06-26T16:41:59.628683548Z", "status": 401, "error": "Unauthorized", "message": "JWT signature is invalid.", "path": "/api/auth/me" }</pre>

"accessToken":

[illegible]

5. Gọi lại POST /api/auth/refresh với refresh token cũ để nhận từ chối.



4.4.3. Kết quả quan sát được

Nhóm quan sát thấy:

- login trả accessToken, refreshToken, role, scope;
- GET /api/auth/me với token hợp lệ trả 200 OK;
- thiếu token hoặc token sai định dạng trả 401;
- token bị sửa payload trả 401 Unauthorized với thông báo chữ ký không hợp lệ;
- refresh token hợp lệ trả cặp token mới;
- refresh token đã logout trả 401 với trạng thái đã bị revoke;
- nếu giảm thời gian sống token trong .env xuống 1 phút, token sẽ bị từ chối sau khi hết hạn.

4.4.4. Giá trị của kịch bản

Về mặt học thuật, đây là kịch bản giúp phân biệt rõ:

- JWT không phải cơ chế “mã hóa nội dung bí mật”;
- chữ ký mới là yếu tố đảm bảo tính toàn vẹn;
- access token và refresh token có vòng đời khác nhau;
- logout trong mô hình stateless thường không phải “xóa session”, mà là revoke refresh token phía server.

4.5. Kịch bản 3: Kiểm chứng luồng quên mật khẩu và đặt lại mật khẩu

4.5.1. Mục tiêu

Kịch bản này chứng minh rằng hệ thống không chỉ dừng ở đăng nhập, mà còn quản lý được vòng đời xác thực tài khoản một cách an toàn. Các điểm cần kiểm chứng gồm:

- người dùng có thể yêu cầu reset password;
- hệ thống sinh reset token;
- trong chế độ demo local, demoResetToken được trả ra để dễ kiểm thử;
- sau khi đổi mật khẩu, refresh token cũ bị revoke;
- response quên mật khẩu không làm lộ email có tồn tại hay không.

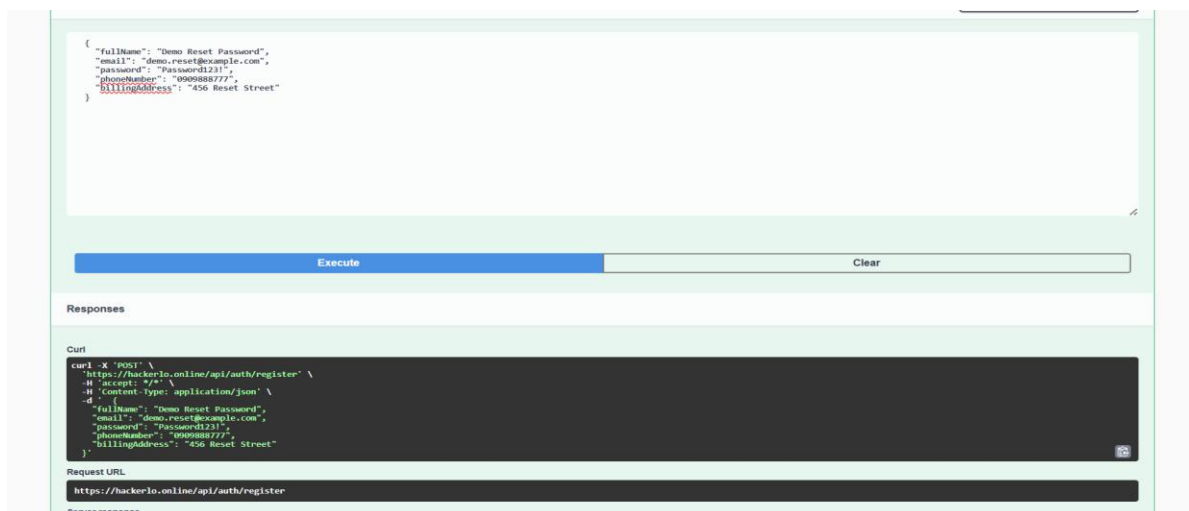
4.5.2. Cách thực hiện

Công cụ nên dùng: Swagger UI hoặc Postman.

Tài khoản nên dùng: một tài khoản phụ mới tạo để tránh làm thay đổi seed data chính.

Nhóm dùng một tài khoản phụ vừa đăng ký hoặc một tài khoản riêng để tránh phá seed data chính. Các bước:

1. Tạo tài khoản mới bằng POST /api/auth/register.



2. Gọi POST /api/auth/forgot-password.

```
{
  "email": "demo.reset@example.com"
}
```

ExecuteClear

Responses

Curl

```
curl -X 'POST' \
  'https://hackerlo.online/api/auth/forgot-password' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "demo.reset@example.com"
  }'
```

Request URL

```
https://hackerlo.online/api/auth/forgot-password
```

Server response

Code	Details
200	Response body <pre>{ "message": "If the email exists, a password reset instruction has been issued.", "demoResetToken": "8dHUEPxcloc5C1qlq5QdNvmerQ5XDykm5m0FeTVkhUYfhs6kE9NcsK38p_RbtJ2fo", "expiresInSeconds": 900 }</pre> Download

3. Gọi POST /api/auth/reset-password.

```
{
  "token": "8dHUEPxcloc5C1qlq5QdNvmerQ5XDykm5m0FeTVkhUYfhs6kE9NcsK38p_RbtJ2fo",
  "newPassword": "NewPassword123!"
}
```

ExecuteClear

Responses

Curl

```
curl -X 'POST' \
  'https://hackerlo.online/api/auth/reset-password' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "token": "8dHUEPxcloc5C1qlq5QdNvmerQ5XDykm5m0FeTVkhUYfhs6kE9NcsK38p_RbtJ2fo",
    "newPassword": "NewPassword123!"
  }'
```

Request URL

```
https://hackerlo.online/api/auth/reset-password
```

Server response

Code	Details
200	Response body <pre>{ "message": "Password reset completed successfully." }</pre> Download

5. Đăng nhập lại bằng mật khẩu cũ.

Request body required application/json

```
{
  "refreshToken": "aDTPde#W0GrkJPauIgf4FCvU3hsIxNcNLSs20y0n_SMdANb1KKz1f00Ey6sJURIOJ"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'https://hackerlo.online/api/auth/refresh' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "refreshToken": "aDTPde#W0GrkJPauIgf4FCvU3hsIxNcNLSs20y0n_SMdANb1KKz1f00Ey6sJURIOJ"
  }'
```

Request URL

```
https://hackerlo.online/api/auth/refresh
```

Server response

Code	Details
401	Error: response status is 401

Response body

```
{
  "timestamp": "7026-06-26T16:32:21.861712794Z",
  "status": 401,
  "error": "Unauthorized",
  "message": "Refresh token has been revoked.",
  "path": "/api/auth/refresh"
}
```

Download

8. Gửi thêm yêu cầu quên mật khẩu với email không tồn tại để so sánh response.

```
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'https://hackerlo.online/api/auth/forgot-password' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "khong-ton-tai@example.com"
  }'
```

Request URL

```
https://hackerlo.online/api/auth/forgot-password
```

Server response

Code	Details
200	

Response body

```
{
  "message": "If the email exists, a password reset instruction has been issued.",
  "demoResetToken": "uj4sLhqXzbjpwq75E3QddGAViyMZ-60DgKNVn-0R4qCh8Vzwz5nXmz9DZUjpi4",
  "expiresInSeconds": 900
}
```

Download

Response headers

4.5.3. Kết quả quan sát được

Kết quả cho thấy:

- endpoint forgot-password trả 200 OK và có demoResetToken trong local demo mode;
- endpoint reset-password trả 200 OK khi token hợp lệ;
- mật khẩu cũ không còn sử dụng được;
- mật khẩu mới sử dụng bình thường;
- refresh token cũ bị từ chối với thông báo đã bị revoke;
- email tồn tại và email không tồn tại đều nhận response có cùng thông điệp; trong local demo mode, response còn giữ cùng cấu trúc với demoResetToken để tránh lộ khác biệt dễ đoán giữa hai trường hợp.

4.5.4. Nhận xét

Kịch bản này có ý nghĩa quan trọng vì nó cho thấy hệ thống không chỉ an toàn ở thời điểm đăng nhập, mà còn an toàn khi khôi phục tài khoản.

4.6. Kịch bản 4: Kiểm chứng mã hóa AES-GCM cho dữ liệu nhạy cảm

4.6.1. Mục tiêu

Kịch bản này nhằm chứng minh:

- dữ liệu nhạy cảm vẫn đọc được qua API khi người dùng hợp lệ;
- cùng dữ liệu đó trong database lại ở dạng ciphertext;
- chỉnh sửa ciphertext sẽ làm giải mã thất bại.

4.6.2. Các dữ liệu được kiểm chứng

Trong dự án hiện tại, các trường được dùng để demo gồm:

- phone_number_encrypted
- billing_address_encrypted
- certificate_code_encrypted

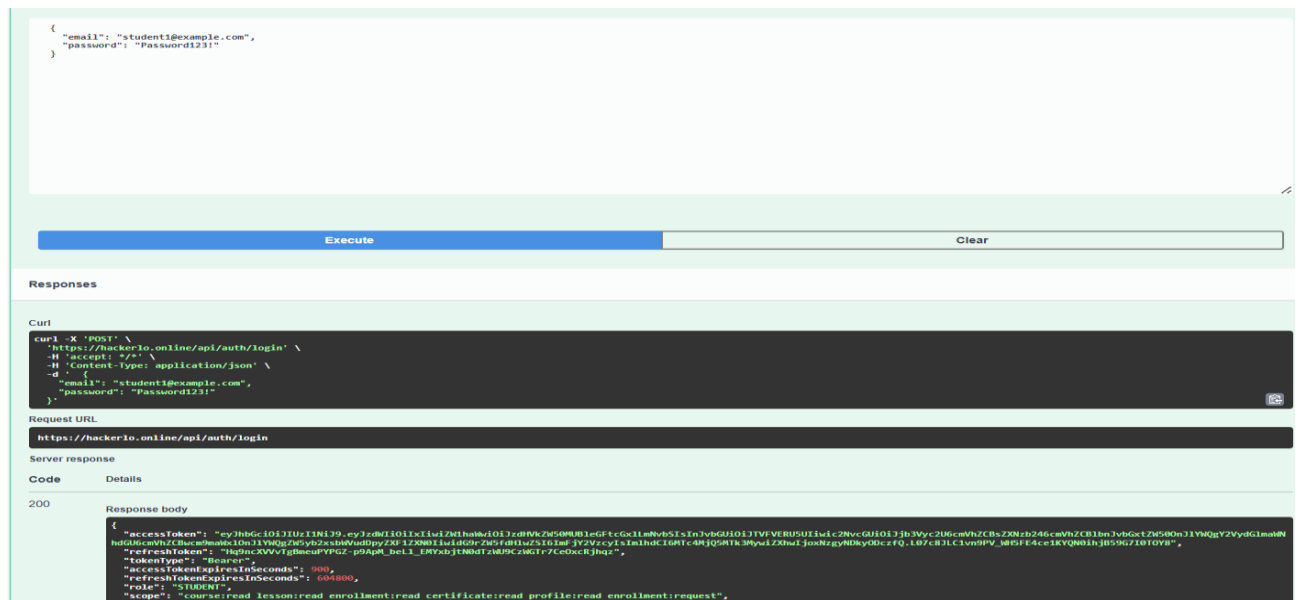
4.6.3. Cách thực hiện

Công cụ nên dùng: Swagger UI để xem dữ liệu đã giải mã ở tầng API, PowerShell để query DB, Maven nếu muốn chạy test tampering.

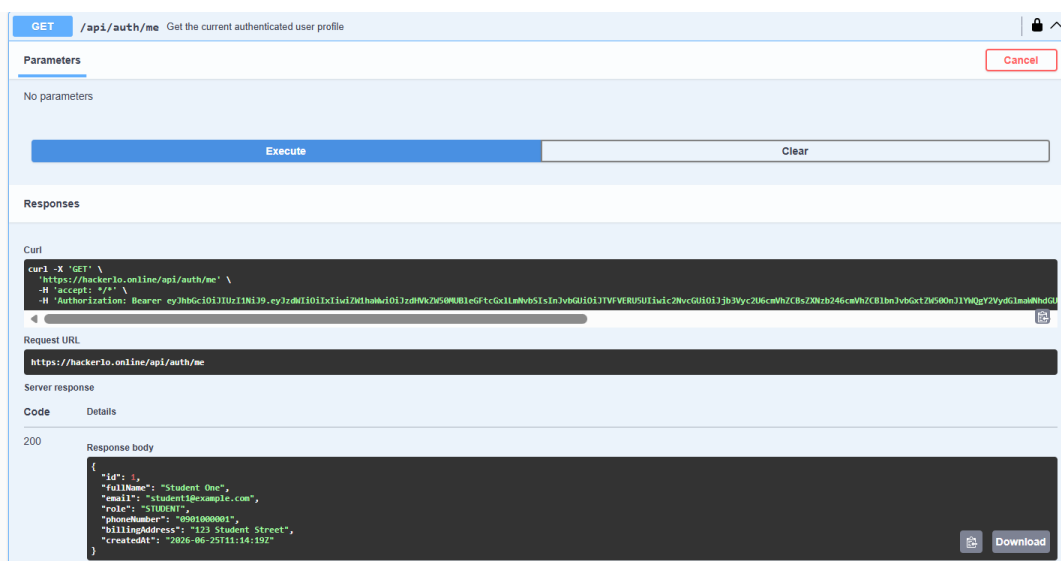
Tài khoản nên dùng: student1@example.com / Password123!.

Các bước chính:

1. Đăng nhập student1 và authorize trên Swagger.



2. Gọi GET /api/auth/me.



3. Gọi GET /api/certificates/me.

GET

/api/certificates/me

List certificates belonging to the current user

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'https://hackerlo.online/api/certificates/me' \
-H 'accept: */*' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6ImlwIiwiaWQiOiJ3ZdWkZlZS00MUB1eGZlcGx1bmVybSIsInR5cGU0Ij01J3VFRVERU5UIiwic2NvcGU0Ij01b3Vyc2U6cmVhZCIsZXNjb246cmVhZCBIbnJvbGctZW50bnJ1YmQgY2VydG1maW5hdGUi
```

Request URL

```
https://hackerlo.online/api/certificates/me
```

Server response

Code

Details

200

Response body

```
{
  "id": 1,
  "ownerUserId": 1,
  "ownerEmail": "student@example.com",
  "courseId": 1,
  "courseTitle": "Java Security Basics",
  "score": 91,
  "certificateCode": "CERT-1-1-FA441332",
  "issuedAt": "2026-06-25T11:14:20Z"
}
```

Download

Response headers

4. Mở PowerShell và truy vấn bảng users, certificates.

```
mysql> SELECT
-> email,phone_number_encrypted,billing_address_encrypted FROM users ORDER BY id;
+-----+-----+-----+
| email | phone_number_encrypted | billing_address_encrypted |
+-----+-----+-----+
| student1@example.com | 1UOAG2130tHtptXZ:3aaNgaEmmwGbvwm==:fKZJf9mXVRE9LX/RRmubHQ== | B7S32U7H1CmMLTAZ:SV3A1rwz6C3WoxpJagsvsvVL:B9GF25mjXmCxi+QyiwPbg== |
| student2@example.com | VNWS0WjLSEyKhky5:D50QAKPnG1JpEA==:zS0syRvcTvkr56bf08XGA== | 5XC10bud8ic1qab:CCP:inxS32z9f1ZzaJfAsXdc:eLDytrIIN4/jgWZEImq1g== |
| admin@example.com | et/4yQKX9fN9F1Yf:1g0f8ATyUmvvaw==:pDYje8BUvbx19ihJNm+zDw== | D8873j10CngLA0mU:h4zVmys5UFErwxYDDYAmoarR8g==:am2FjLNC49axNea8weUgw== |
| postman.student@example.com | aTWHL0JHtL3gLL:1BC/RmGpDi8iYQ==:eFSaQ6iFQkThIdg8EJXSQ0== | 9VC19zZXMuI/X4c4:t1eKASJRvU40TN87YfzXGLa7:uSRBSThMS1p62Q+F4YKdVA== |
| studentman1.student@example.com | BBHmJmCbCrmGHycE:qcaXnm3PH8hwm==:Y0rUhsps1aG4mb6wbe1J3g== | GwSkLwX7ahm96907:038yf3P0mU8oeiepZxqL15U+1:kucdRqBpAuTjNnq36i1ZBg== |
| demo.reset@example.com | 93qcq/wtr8HeBzOV:AR3R3C9wxUV+ew==:BvvIhaOGZ13vSy3ogk8t1g== | tkUXMQP+cRDHW/HT:4s8H40G6a1FdmNR0oFxFhg==:SYNAr6t1VCVPwa82oMiTGw== |
+-----+-----+-----+

6 rows in set (0.00 sec)

mysql> SELECT
-> id,student_id,course_id,certificate_code_encrypted,score FROM certificates ORDER BY id;
+-----+-----+-----+-----+-----+
| id | student_id | course_id | certificate_code_encrypted | score |
+-----+-----+-----+-----+-----+
| 1 | 1 | 1 | 1V4S08gSyWAG8iYc:+y9gv6u4Vm3shhgjG0uxPsTs==:++12niVz31FIQjySEqrVka== | 91.50 |
| 2 | 1 | 2 | JtPwkhK7+Pb3b8RM:Lpt7bqDABdWf20URGi1uYQ==:dbX/OuV9MmHiB9IpenCuYQ== | 95.00 |
+-----+-----+-----+-----+-----+

2 rows in set (0.00 sec)

mysql> |
```

5. Chạy EncryptionServiceTest để kiểm chứng tampering fail.

```
PS E:\PROJECT_MMUD\backend> mvn -Dtest=EncryptionServiceTest test
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.company:securityapp >-----
[INFO] Building securityapp 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ securityapp ---
[INFO] Copying 3 resources from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ securityapp ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ securityapp ---
[INFO] skip non existing resourceDirectory E:\PROJECT_MMUD\backend\src\test\resources
[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ securityapp ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ securityapp ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.company.securityapp.EncryptionServiceTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.149 s -- in com.company.securityapp.EncryptionServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.527 s
[INFO] Finished at: 2026-06-26T23:55:18+07:00
[INFO]
[INFO] -----
PS E:\PROJECT_MMUD\backend>
```

4.6.4. Kết quả quan sát được

Kết quả cho thấy:

- phía API, các trường phoneNumber, billingAddress, certificateCode hiển thị ở dạng có nghĩa;
- phía database, các cột tương ứng lưu chuỗi ciphertext không đọc được;
- test aesGcmRoundTripWorksAndTamperingFails chạy thành công, chứng minh sửa ciphertext sẽ dẫn tới lỗi giải mã.

4.6.5. Đánh giá

Kịch bản này giúp phân biệt rất rõ:

- dữ liệu cần đọc lại dùng AES-GCM;
- mật khẩu không dùng AES-GCM mà dùng bcrypt;
- tính toàn vẹn của AES-GCM là một giá trị thực tế chứ không chỉ là khái niệm lý thuyết.

4.7. Kịch bản 5: Chống BOLA/IDOR và kiểm soát quyền trên tài nguyên

4.7.1. Mục tiêu

Kịch bản này là minh chứng trọng tâm cho bảo mật API vì:

- chỉ xác thực thôi là chưa đủ;
- phải kiểm tra quyền trên từng object;
- việc chặn phải diễn ra ở backend.

4.7.2. Tình huống thực nghiệm chính

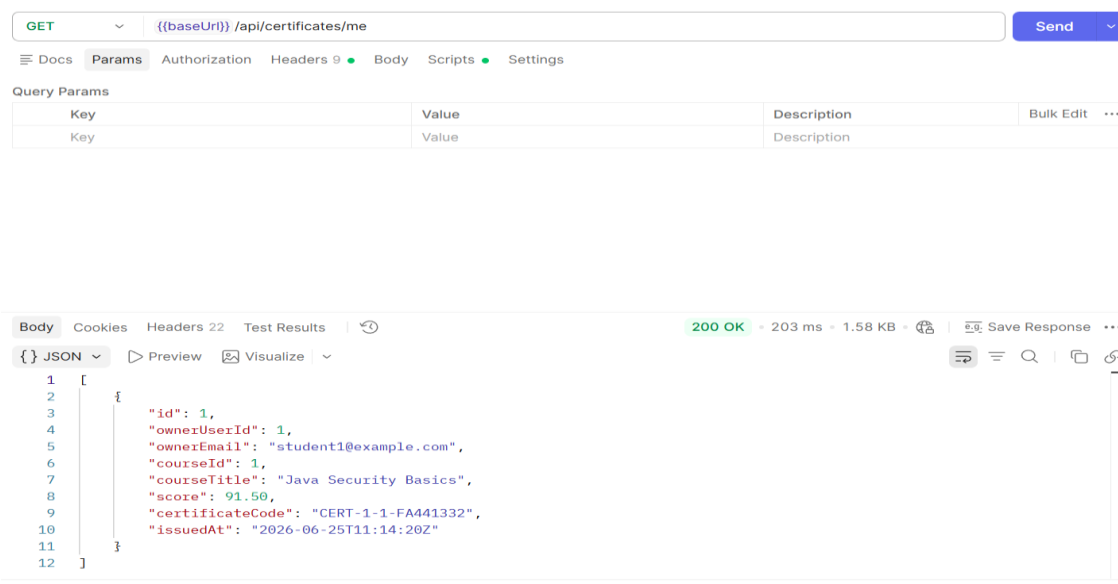
Công cụ nên dùng: Postman là thuận tiện nhất vì giữ được token của student1, student2 và admin cùng lúc.

Tài khoản nên dùng: student1@example.com, student2@example.com, admin@example.com.

Dữ liệu cần chuẩn bị: lấy trước certificateId của student1, và nếu demo lesson thì chuẩn bị thêm courseId, lessonId chưa có enrollment ACTIVE.

Dùng chứng chỉ của student1 làm tài nguyên mục tiêu và thực hiện:

1. student1 gọi GET /api/certificates/me để lấy certificateId.



2. student2 dùng token của mình để gọi GET /api/certificates/{certificateId}.

The screenshot shows a Postman interface for a GET request to the endpoint `{{baseUrl}}/api/certificates/{{victimCertificateId}}`. The response is a 403 Forbidden status with a JSON body containing the following details:

```
{
  "timestamp": "2026-06-26T17:10:32.690641597Z",
  "status": 403,
  "error": "Forbidden",
  "message": "Only the owning student can access this certificate.",
  "path": "/api/certificates/1"
}
```

3. admin mở GET /api/admin/audit-logs để xác nhận hành vi bị chặn đã được ghi nhận.

The screenshot shows a Postman interface for a GET request to the endpoint `{{baseUrl}}/api/admin/audit-logs`. The response is a 200 OK status with a JSON body containing the following details:

```
{
  "id": 133,
  "actorUserId": 2,
  "actorEmail": "student2@example.com",
  "action": "ACCESS_DENIED",
  "targetType": "Certificate",
  "targetId": 1,
  "ipAddress": "14.169.31.120",
  "userAgent": "PostmanRuntime/1.8.3",
  "status": "FAILED",
  "message": "Only the owning student can access this certificate.",
  "createdAt": "2026-06-26T17:10:33Z"
},
```

4.7.3. Kết quả quan sát được

Kết quả thực tế:

- Chủ sở hữu certificate xem được dữ liệu của mình;
- Student2 bị trả 403 Forbidden khi đổi certificateId;
- Lesson bị khóa trả 403 cho người chưa đủ điều kiện;
- Audit log có ACCESS_DENIED.

4.7.4. Đánh giá

Đây là một trong những phần có giá trị nhất của đề án vì nó gắn trực tiếp với OWASP API: Broken Object Level Authorization. Bởi vì nó có:

- Có thao tác tấn công cụ thể;
- Có kết quả chặn cụ thể;
- Có log giám sát cụ thể.

4.8. Kịch bản 6: Yêu cầu ghi danh và phê duyệt ghi danh

4.8.1. Mục tiêu

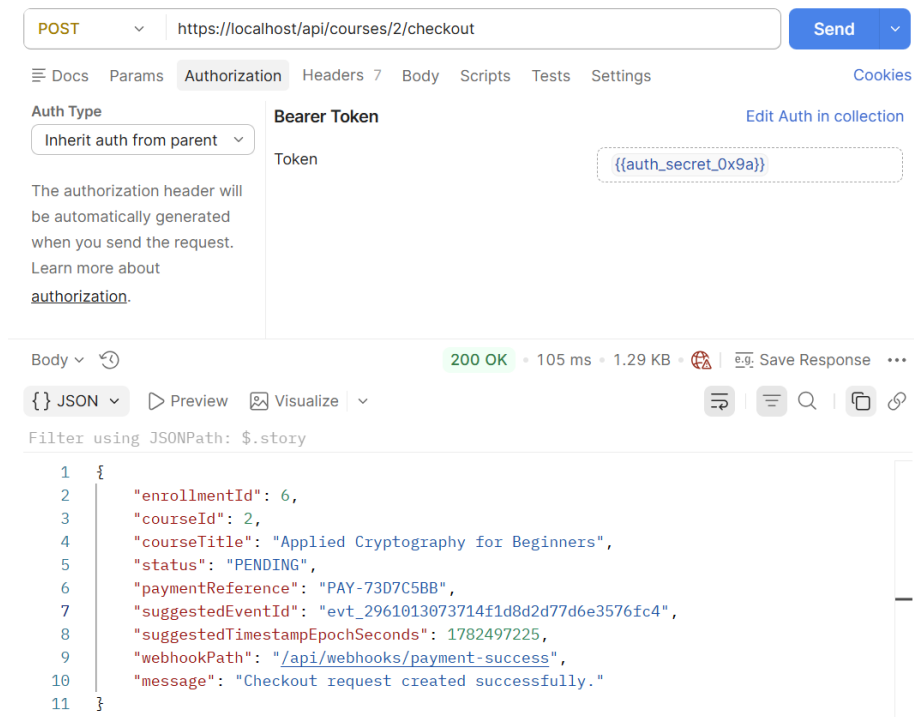
Kịch bản này minh họa rất rõ cách bảo mật API gắn với logic nghiệp vụ thực tế. Những điểm cần chứng minh:

- sinh viên chỉ tạo được enrollment ở trạng thái PENDING;
- quản trị viên nhìn thấy danh sách pending request của từng khóa học;
- chỉ ADMIN mới có thể phê duyệt enrollment thành ACTIVE;
- bài học chỉ mở khóa sau khi enrollment đã được duyệt.

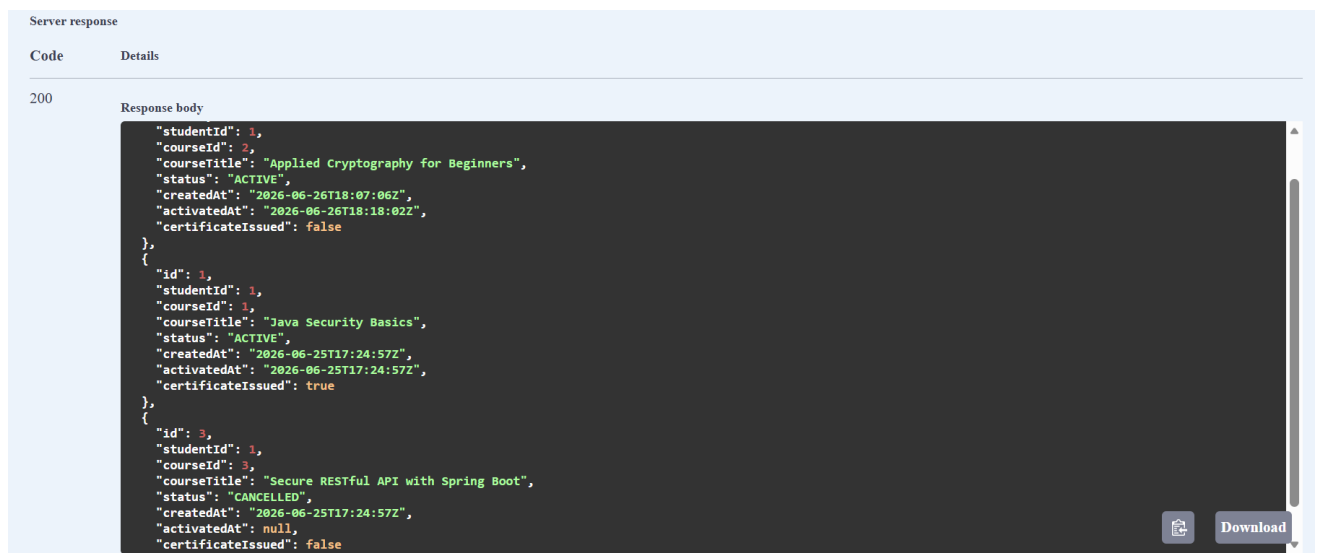
4.8.2. Cách thực hiện

Chuỗi thao tác:

1. student1 gọi POST /api/courses/{courseId}/enrollment-requests.



2. Kiểm tra GET /api/enrollments/me để xác nhận enrollment vừa tạo có trạng thái PENDING.



3. admin gọi GET `/api/admin/courses/{courseId}/enrollments` để thấy request đang chờ duyệt.

Server response

Code

Details

200

Response body

```
{
  "courseId": 2,
  "courseTitle": "Applied Cryptography for Beginners",
  "courseSummary": "AES-GCM, HMAC-SHA256, and secure key handling for students.",
  "price": 249000,
  "instructorName": "Admin Demo",
  "activeStudentCount": 2,
  "pendingRequestCount": 1,
  "activeStudents": [
    {
      "enrollmentId": 4,
      "studentId": 4,
      "studentName": "Nguyen Chi",
      "studentEmail": "huuchi@example.com",
      "courseId": 2,
      "courseTitle": "Applied Cryptography for Beginners",
      "status": "ACTIVE",
      "createdAt": "2026-06-25T17:30:59Z",
      "activatedAt": "2026-06-25T17:31:30Z",
      "certificateIssued": false
    },
    {
      "enrollmentId": 2,
      "studentId": 2,
      "studentName": "Student Two",
      "studentEmail": "student2@example.com",
      "courseId": 2,

```

4. admin gọi POST /api/admin/enrollments/{enrollmentId}/approve.

```
Curl
curl -X 'POST' \
  'https://localhost/api/admin/enrollments/6/approve' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWQ6IiwiaWF0Ij01ZD9iY291cnNlOnJlYWQ6ZW5yZSxbWVduDptYWhShZUgdXNl' \
  -d ''

Request URL
https://localhost/api/admin/enrollments/6/approve

Server response
Code    Details

200
Response body
{
  "enrollmentId": 6,
  "studentId": 1,
  "studentName": "Student One",
  "studentEmail": "student1@example.com",
  "courseId": 2,
  "courseTitle": "Applied Cryptography for Beginners",
  "status": "ACTIVE",
  "createdAt": "2026-06-26T18:07:06Z",
  "activatedAt": "2026-06-26T18:18:01.766886394Z",
  "certificateIssued": false
}
```

5. student1 gọi lại GET /api/enrollments/me hoặc GET /api/courses/{courseId}/lessons/{lessonId} để kiểm tra trạng thái sau khi duyệt.

Request URL

https://localhost/api/courses/2/lessons/3

Server response

CodeDetails

200

Response body

```
{
  "id": 3,
  "courseId": 2,
  "courseTitle": "Applied Cryptography for Beginners",
  "title": "AES-GCM for Data at Rest",
  "previewText": "Encrypt sensitive fields with IV and authentication tag.",
  "content": "AES-GCM for Data at Rest\n\nThis lesson contains the unlocked course content used during the security demo.",
  "sortOrder": 1,
  "unlocked": true
}
```

Download

4.8.3. Kết quả quan sát được

Khi sinh viên vừa gửi yêu cầu ghi danh:

- enrollment ở trạng thái PENDING;
- lesson vẫn chưa được mở khóa;
- nếu gọi trực tiếp lesson protected thì backend vẫn trả 403.

Khi admin mở danh sách của khóa học:

- pending request xuất hiện trong khu vực quản trị;
- admin có thể phân biệt rõ danh sách đang học và danh sách chờ duyệt.

Khi admin duyệt thành công:

- response trả về 200 OK;
- enrollment được chuyển sang ACTIVE;
- lesson mở khóa cho student và có thể trả nội dung đầy đủ.

4.8.4. Đánh giá

Kịch bản này cho thấy chỉ có JWT là chưa đủ. Muốn bảo vệ tài nguyên học tập đúng cách, hệ thống còn phải ràng buộc thêm role và trạng thái nghiệp vụ. Nếu thiếu bước phê duyệt hoặc kiểm tra ACTIVE, sinh viên có thể truy cập lesson sớm hơn quyền được cấp.

4.9. Kịch bản 7: Rate limiting và ghi nhận hành vi bất thường

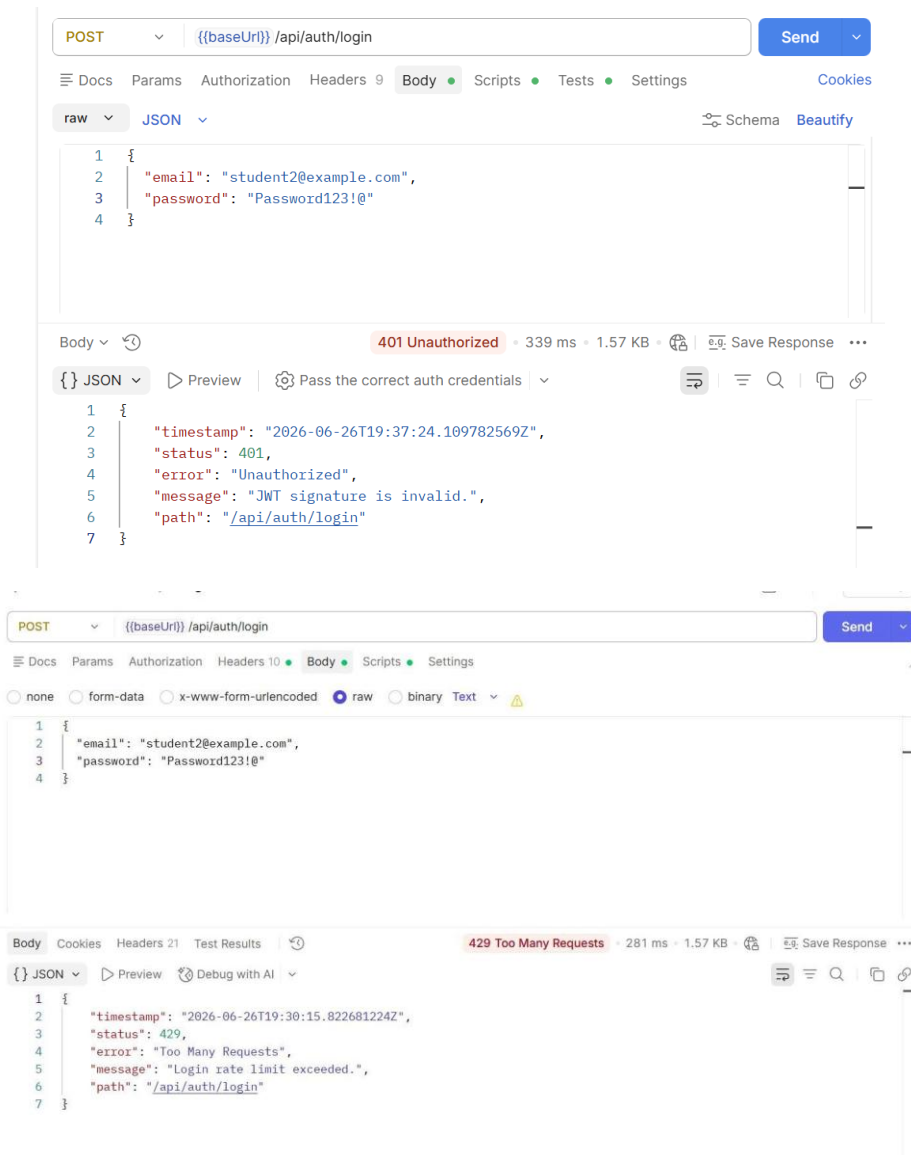
4.9.1. Mục tiêu

Kịch bản này nhằm chứng minh hệ thống có khả năng giảm brute-force và spam API bằng cách:

- đếm số request trong cửa sổ thời gian;
- chặn khi vượt ngưỡng;
- ghi log phục vụ giám sát.

4.9.2. Cách thực hiện

Dùng PowerShell gửi liên tiếp nhiều request sai mật khẩu tới: POST /api/auth/login



The first screenshot shows a POST request to `{{baseUrl}}/api/auth/login` with a JSON body containing email and password. The response is a 401 Unauthorized status with a message indicating the JWT signature is invalid.

The second screenshot shows the same POST request, but the response is a 429 Too Many Requests status, indicating that the login rate limit has been exceeded.

```
{
  "id": 146,
  "actorUserId": null,
  "actorEmail": null,
  "action": "RATE_LIMIT_EXCEEDED",
  "targetType": "Request",
  "targetId": null,
  "ipAddress": "14.169.31.120",
  "userAgent": "PostmanRuntime/1.8.3",
  "status": "FAILED",
  "message": "Login rate limit exceeded.",
  "createdAt": "2026-06-26T19:30:16Z"
},
```

4.9.3. Kết quả quan sát được

Kết quả có mẫu rất rõ:

- vài request đầu trả 401 Unauthorized;
- sau khi vượt ngưỡng, hệ thống trả 429 Too Many Requests;
- audit log có RATE_LIMIT_EXCEEDED.

4.9.4. Đánh giá

Mặc dù cơ chế hiện tại mới dùng bộ nhớ trong tiến trình, nhưng đối với mục tiêu đồ án, đây là giải pháp phù hợp vì: dễ hiểu; dễ chứng minh; dễ demo; cho thấy rõ ý nghĩa của phòng thủ trước tấn công tự động.

4.10. Kịch bản 8: HTTPS/TLS, Swagger/OpenAPI và quét OWASP ZAP

4.10.1. Mục tiêu

Kịch bản này kết hợp ba lớp thực nghiệm:

- chứng minh HTTP bị ép sang HTTPS;
- chứng minh Swagger/OpenAPI có thể dùng để test hệ thống protected;
- chứng minh hệ thống đã chạy quét baseline bằng OWASP ZAP.

4.10.2. Kết quả kiểm tra HTTPS/TLS

Khi chạy:

```
curl.exe -I http://localhost/api/health  
curl.exe -k https://localhost/api/health
```

Ta thấy được:

```
PS E:\PROJECT_MMUD> curl.exe -I http://localhost/api/health  
HTTP/1.1 301 Moved Permanently  
Server: nginx  
Date: Fri, 26 Jun 2026 19:54:25 GMT  
Content-Type: text/html  
Content-Length: 162  
Connection: keep-alive  
Location: https://localhost/api/health
```

- request HTTP trả 301 Moved Permanently

```
PS E:\PROJECT_MMUD> curl.exe -k -i https://localhost/api/health
HTTP/1.1 200
Server: nginx
Date: Fri, 26 Jun 2026 20:00:15 GMT
Content-Type: application/json
Transfer-Encoding: chunked
Connection: keep-alive
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(), microphone=(), geolocation=()
Strict-Transport-Security: max-age=31536000; includeSubDomains
```

```
○ {"status":"UP","service":"securityapp-backend","activeProfiles":["dev"]}PS E:\PROJECT_MMUD> █
```

- request HTTPS trả 200 OK.

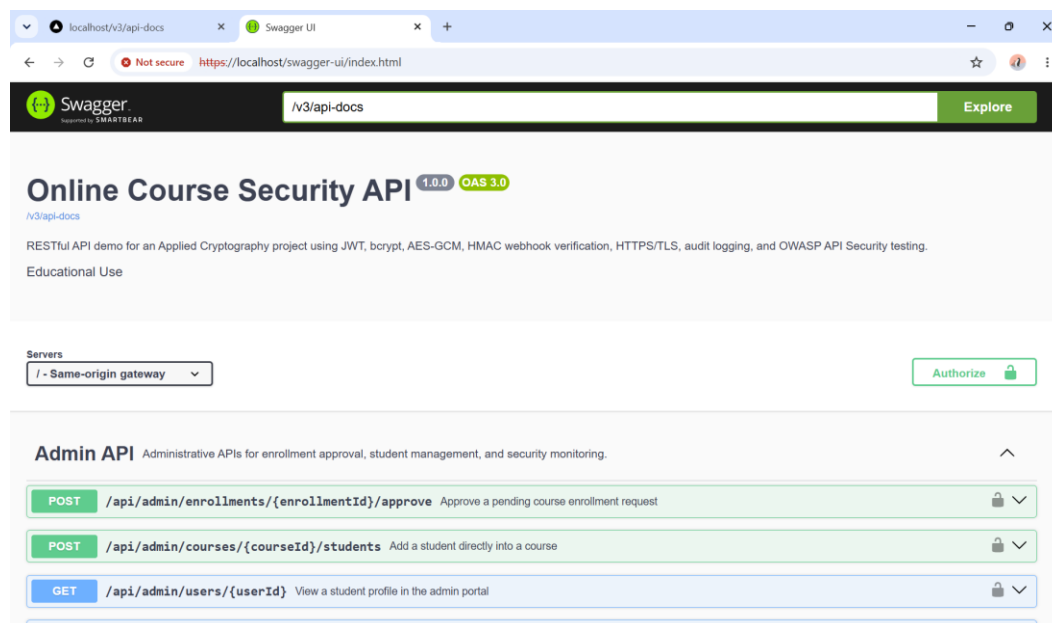
Điều này cho thấy reverse proxy đã buộc sử dụng kênh truyền an toàn.

4.10.3. Kết quả kiểm tra Swagger/OpenAPI

Khi mở:

- <https://localhost/swagger-ui.html>
- <https://localhost/v3/api-docs>

Swagger phản hồi ổn định và có thể nhập Bearer token để kiểm thử endpoint protected.



Summary of Alerts trong file HTML cho thấy:

- High = 0
- Medium = 4
- Low = 0
- Informational = 1

4.10.5. Đánh giá

Kịch bản này làm rõ thêm bức tranh bảo mật của đồ án ở tầng triển khai:

- TLS bảo vệ dữ liệu khi truyền;
- Swagger hỗ trợ kiểm thử có kiểm soát;
- ZAP cung cấp thêm một bằng chứng khách quan từ công cụ DAST phổ biến.

4.11. Đối chiếu với các tiêu chí chấp nhận của đề tài

Để đánh giá tổng thể, bảng sau đối chiếu trực tiếp trạng thái hệ thống với các yêu cầu cốt lõi đã đặt ra.

Tiêu chí	Trạng thái	Minh chứng
Backend Spring Boot chạy được	Đạt	Truy cập https://localhost/api/health
Frontend NextJS chạy được	Đạt	Truy cập https://localhost
Đăng ký và đăng nhập hoạt động	Đạt	Swagger, Postman, integration test
Mật khẩu lưu dạng bcrypt	Đạt	Query bảng users , Demo 01
JWT dùng được cho protected API	Đạt	GET /api/auth/me , Demo 02

Token sai hoặc hết hạn bị từ chối	Đạt	Demo 02, integration test
Có role tách biệt STUDENT và ADMIN	Đạt	SecurityConfig , Demo RBAC
Student không xem được dữ liệu của student khác	Đạt	Demo BOLA/IDOR
Có mã hóa AES-GCM cho dữ liệu nhạy cảm	Đạt	Query DB, EncryptionServiceTest
Có luồng yêu cầu ghi danh riêng cho STUDENT	Đạt	POST /api/courses/{courseId}/enrollment-requests , Demo 06
Enrollment PENDING chưa mở lesson	Đạt	Hình 10, Demo 06
Admin duyệt enrollment thành ACTIVE	Đạt	POST /api/admin/enrollments/{enrollmentId}/approve , Hình 13
Có audit log	Đạt	GET /api/admin/audit-logs
Admin xem được audit log	Đạt	Demo admin
Có rate limit cho login/password reset/admin	Đạt	429 Too Many Requests
Có Swagger/OpenAPI	Đạt	swagger-ui.html , v3/api-docs

Swagger hỗ trợ Bearer JWT	Đạt	Nút Authorize và thử endpoint protected
Có Docker và Docker Compose	Đạt	docker-compose.yml và stack đang chạy
Có .env.example	Đạt	Kiểm tra file cấu hình của dự án
Có tài liệu Postman và ZAP	Đạt	docs/postman-testing.md , docs/owasp-zap-testing.md

4.12. Đánh giá tổng hợp về giá trị học thuật và thực hành

Từ các kịch bản thực nghiệm ở trên, có thể rút ra một số đánh giá tổng hợp như sau.

Thứ nhất, đề tài đạt được các giá trị học thuật vì đã làm rõ được sự khác nhau về vai trò giữa các cơ chế mật mã:

- bcrypt bảo vệ mật khẩu;
- AES-GCM bảo vệ dữ liệu cần đọc lại;
- JWT dùng cho xác thực request;
- TLS bảo vệ dữ liệu khi truyền.

Thứ hai, đề tài đạt được các giá trị thực hành vì mỗi kỹ thuật đều có vị trí cụ thể trong luồng nghiệp vụ:

- đăng ký và đăng nhập;
- quên mật khẩu;
- gửi yêu cầu ghi danh và phê duyệt ghi danh;
- xem bài học;
- xem chứng chỉ;
- quản trị enrollment;
- giám sát audit log.

Chương 5. Kết luận và kiến nghị

5.1. Tóm tắt kết quả toàn đề tài

Đề tài đã xây dựng được một hệ thống hoàn chỉnh ở mức demo kỹ thuật. Trên nền tảng công nghệ đã lựa chọn gồm Spring Boot, Spring Security, NextJS, MySQL, Nginx và Docker Compose, nhóm tổ chức hệ thống theo hướng web bán khóa học online tối giản và tích hợp các kỹ thuật mật mã ứng dụng một cách có chủ đích, để mỗi cơ chế bảo vệ đều gắn với một tình huống nghiệp vụ và một kịch bản kiểm chứng cụ thể.

Các kết quả nổi bật có thể tóm tắt như sau:

- triển khai backend Spring Boot và frontend NextJS chạy qua HTTPS;
- bảo vệ xác thực bằng JWT access token và refresh token;
- xây dựng các luồng logout, quên mật khẩu và đặt lại mật khẩu;
- băm mật khẩu bằng bcrypt;
- mã hóa dữ liệu nhạy cảm bằng AES-GCM;
- xây dựng luồng STUDENT gửi yêu cầu ghi danh và ADMIN phê duyệt rõ ràng;
- chặn BOLA/IDOR bằng kiểm tra ownership phía server;
- áp dụng rate limiting cho các endpoint nhạy cảm;
- ghi audit log cho các sự kiện bảo mật;
- hỗ trợ kiểm thử bằng Swagger, Postman, script PowerShell và OWASP ZAP.

5.2. Đóng góp chính của đề tài

5.2.1. Đóng góp về mặt kỹ thuật

Đề tài đã thực hiện được một mô hình bảo mật API nhiều lớp đơn giản. Điểm mạnh của sản phẩm không phải là độ lớn của hệ thống, mà là việc mỗi lớp bảo vệ đều được gắn phù hợp:

- bcrypt tại lớp lưu mật khẩu;
- JWT tại lớp xác thực request;

- AES-GCM tại lớp dữ liệu lưu trữ;
- trạng thái PENDING/ACTIVE kết hợp role ở lớp logic ghi danh;
- TLS tại lớp truyền thông;
- RBAC và ownership check tại lớp phân quyền;
- rate limit và audit log tại lớp giám sát và phòng thủ.

5.2.2. Đóng góp về mặt học thuật

Đề tài giúp làm rõ một số nhầm lẫn phổ biến khi học mật mã ứng dụng:

- không phải dữ liệu nhạy cảm nào cũng dùng cùng một kỹ thuật bảo vệ;
- không thể dùng “mã hóa” thay cho “băm mật khẩu”;
- JWT không tự động an toàn nếu backend không kiểm tra đầy đủ;
- chỉ xác thực thành công chưa có nghĩa là được quyền truy cập mọi tài nguyên;
- logic nghiệp vụ cũng là một bề mặt tấn công thực sự nếu không gắn role và trạng thái đúng cách.

5.2.3. Đóng góp về mặt trình bày đồ án

Đề tài có nhiều đầu tư thử nghiệm ở khía cạnh demo và báo cáo:

- mỗi kỹ thuật đều có endpoint hoặc tình huống demo riêng;
- có thể dùng cả Swagger, Postman và PowerShell;
- có tài liệu ảnh, caption, runbook và hướng dẫn kiểm thử;

5.3. Ý nghĩa của đề tài đối với môn học

Học phần không chỉ dừng ở việc biết tên thuật toán, mà yêu cầu người học hiểu:

- khi nào dùng thuật toán nào;
- dữ liệu nào cần băm, dữ liệu nào cần mã hóa;
- dữ liệu nào cần ký hoặc dữ liệu nào xác thực toàn vẹn;
- các khóa bí mật cần được quản lý như thế nào;
- những sai lầm phổ biến khi đưa thuật toán vào hệ thống thật là gì.

Đề tài này góp phần làm rõ các câu hỏi trên thông qua một ứng dụng thu gọn nhưng đủ thực tiễn. Cụ thể:

- bcrypt được dùng để lưu mật khẩu, không phải để lưu dữ liệu hồ sơ;
- AES-GCM được dùng cho số điện thoại, địa chỉ và mã chứng chỉ;
- JWT dùng chữ ký đối xứng để bảo vệ tính toàn vẹn token;
- JWT dùng để mang claim xác thực và phân quyền;
- JWT_SECRET và ENCRYPTION_KEY được tách ra biến môi trường thay vì hard-code.

5.4. Hạn chế của đề tài

Mặc dù đã đạt mục tiêu chính, đề tài vẫn còn một số hạn chế cần nhìn nhận thẳng thắn.

Thứ nhất, rate limiting hiện được xây dựng theo kiểu lưu bộ đếm trong bộ nhớ tiến trình, phù hợp để demo local nhưng chưa tối ưu cho hệ thống phân tán nhiều instance.

Thứ hai, certificate local đang dùng self-signed certificate. Điều này đủ cho thực nghiệm và kiểm thử, nhưng không đại diện cho cấu hình production với chứng chỉ công khai hợp lệ.

Thứ ba, frontend được tối giản để phục vụ demo bảo mật, nên chưa áp dụng các chiến lược lưu token tối ưu nhất cho triển khai thực tế quy mô lớn.

Thứ tư, chế độ PASSWORD_RESET_DEMO_MODE giúp việc kiểm thử dễ hơn nhưng không phù hợp khi triển khai production, vì reset token không nên trả trực tiếp qua API cho người dùng.

Thứ năm, luồng ghi danh hiện mới dừng ở mức admin duyệt trực tiếp trên hệ thống; chưa có thông báo email, hàng đợi xử lý hoặc quy trình phê duyệt nhiều bước cho môi trường thực tế lớn hơn.

Thứ sáu, đề tài chưa triển khai các cơ chế nâng cao như:

- key rotation tự động;
- secret manager hoặc KMS;
- giám sát thời gian thực, SIEM hoặc cảnh báo tập trung;
- kiểm thử bảo mật chuyên sâu hơn ở quy mô production.

5.5. Hướng phát triển trong tương lai

Nếu tiếp tục mở rộng đề tài sau môn học, nhóm phát triển đề án theo các hướng sau:

1. Chuyển rate limiting sang Redis để dùng được cho nhiều instance backend.
2. Chuyển cơ chế lưu token của frontend sang mô hình an toàn hơn, ví dụ HTTP-only cookie kết hợp CSRF protection phù hợp ngữ cảnh.
3. Tích hợp cơ chế key rotation cho JWT secret và encryption key.
4. Bổ sung thông báo email hoặc cơ chế hàng đợi khi enrollment được phê duyệt.
5. Xây dựng dashboard giám sát audit log và cảnh báo bất thường.
6. Xây dựng CI/CD kèm SAST, dependency scanning và DAST tự động.
7. Mở rộng vai trò nghiệp vụ nếu cần, ví dụ tách INSTRUCTOR riêng ở mức đầy đủ hơn, nhưng vẫn phải giữ nguyên định hướng bảo mật là trung tâm.
8. Bổ sung cơ chế tự động duyệt ghi danh sau khi người học thanh toán thành công thông qua mã QR.
9. Hoàn thiện tính năng quên mật khẩu bằng cách gửi liên kết hoặc mã xác thực đặt lại mật khẩu về email của người dùng.

Tài liệu tham khảo

- [1] OWASP Foundation, *OWASP API Security Top 10 - 2023*, 2023.
- [2] OWASP Foundation, *Password Storage Cheat Sheet*, OWASP Cheat Sheet Series.
- [3] OWASP Foundation, *Cryptographic Storage Cheat Sheet*, OWASP Cheat Sheet Series.
- [4] National Institute of Standards and Technology, *Digital Identity Guidelines: Authentication and Authenticator Management (NIST SP 800-63B-4)*, July 2025.
- [5] M. Jones, J. Bradley và N. Sakimura, *RFC 7519: JSON Web Token (JWT)*, Internet Engineering Task Force, May 2015.
- [6] Y. Sheffer, D. Hardt và M. Jones, *RFC 8725: JSON Web Token Best Current Practices*, Internet Engineering Task Force, February 2020
- [7] M. Jones và D. Hardt, *RFC 6750: The OAuth 2.0 Authorization Framework: Bearer Token Usage*, Internet Engineering Task Force, October 2012.
- [8] National Institute of Standards and Technology, *SP 800-38D: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, November 2007.
- [9] H. Krawczyk, M. Bellare và R. Canetti, *RFC 2104: HMAC: Keyed-Hashing for Message Authentication*, Internet Engineering Task Force, February 1997.
- [10] OWASP Foundation, *Transport Layer Security Cheat Sheet*, OWASP Cheat Sheet Series.
- [11] Spring Team, *Spring Security Reference Documentation*, tài liệu kỹ thuật chính thức.
- [12] OpenAPI Initiative, *OpenAPI Specification v3.2.0*, 19 September 2025.
- [13] OWASP ZAP Project, *ZAP Documentation*, tài liệu chính thức của công cụ quét bảo mật.

